



**RAJALAKSHMI
ENGINEERING
COLLEGE**

B.Tech

COMPUTER SCIENCE AND BUSINESS SYSTEMS

CB23731

**Data Visualization Techniques
(Regulation 2023)**

LAB RECORD

ACADEMIC YEAR: 2026-2027 (ODD)

Name

Register No. :

Year / Branch / Section

Semester

Academic Year :



**RAJALAKSHMI
ENGINEERING
COLLEGE**

BONAFIDE CERTIFICATE

NAME.....

ACADEMIC YEAR SEMESTER BRANCH.....

UNIVERSITY REGISTER NO.

Certified that this is the bonafide record of work done by the
above student during the year 20 - 20

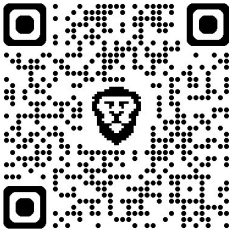
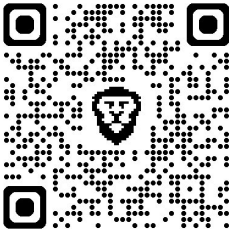
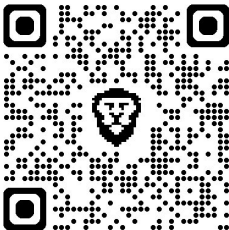
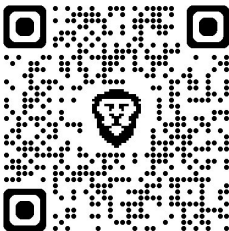
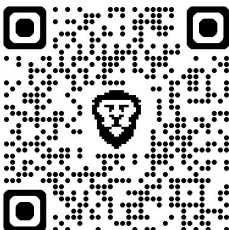
Signature of Faculty-in-charge

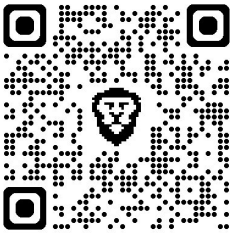
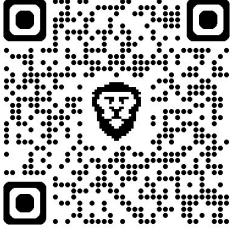
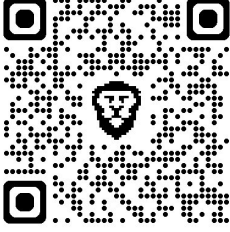
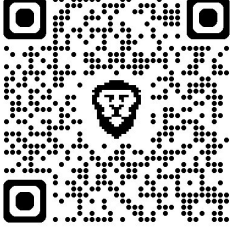
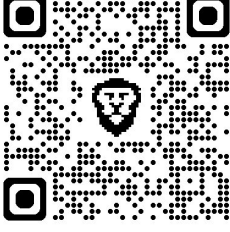
Submitted for the Practical Examination held on

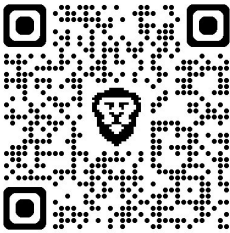
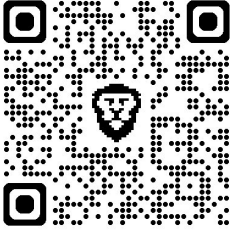
Internal Examiner

External Examiner

INDEX

Date	Experiment No.	Experiment Title	QR Code	Sign
	1a	Setting Up Python Environment and Libraries – Jupyter Notebook		
	1b	EDA – Data Cleaning		
	2	EDA – Data Import and Export		
	3	Data Cleaning		
	4(a)	Exploratory Data Analysis (EDA) – Data Inspection, Filtering, Statistical Analysis And Correlation		

	5	Data Visualization Using Matplotlib		
	6a	Data Visualization Using Power Bi		
	6b	Data Visualization Using Power BI – Advanced Sales Analytics		
	6c	Create a Dashboard in Power Bi		
	7a	Create Dashboard Using Tableau With Student Marksheet Dataset		
	7b	Create Dashboard Using Tableau		

	8	Mini Project – Social Media Analytics		
	9	Mini Project – Healthcare Analytics		

EX:1a

SETTING UP PYTHON ENVIRONMENT AND LIBRARIES – JUPYTER NOTEBOOK

Aim:

To install Python, configure Jupyter Notebook, understand the notebook environment, execute Python programs, create Markdown cells, and use basic Jupyter Notebook features for Data Analytics applications.

Algorithm:

1. Start the program.
2. Import the required libraries (if needed).
3. Declare and initialize the required variables or data.
4. Perform the required operations or processing.
5. Display the output/results.
6. Stop the program.

Code:

```
#To print  
print("Welcome to Data Analytics Lab")
```

Welcome to Data Analytics Lab

```
#declare variable and perform mathematical functions  
a = 20  
b = 10  
print(a+b)  
print(a-b)  
print(a*b)  
print(a/b)
```

```
30  
10  
200  
2.0
```

```
#declare variables and print it
```

```
name = "John"
```

```
dept = "AI&DS"
```

```
cgpa = 8.9
```

```
print("Name:", name)
```

```
print("Department:", dept)
```

```
print("CGPA:", cgpa)
```

Name: John

Department: AI&DS

CGPA: 8.9

```
#import numpy and perform mean and sum operations
```

```
import numpy as np
```

```
a = np.array([10,20,30,40,50])
```

```
print(a)
```

```
print(np.mean(a))
```

```
print(np.sum(a))
```

[10 20 30 40 50]

30.0

150

```
#import pandas as pd and create a dcitionary
```

```
#and make it as a DataFrame
```

```
import pandas as pd
```

```
data = {
```

```
    'Name': ['John', 'David', 'Alex'],
```

```
    'Marks': [85, 90, 95]
```

```
}
```

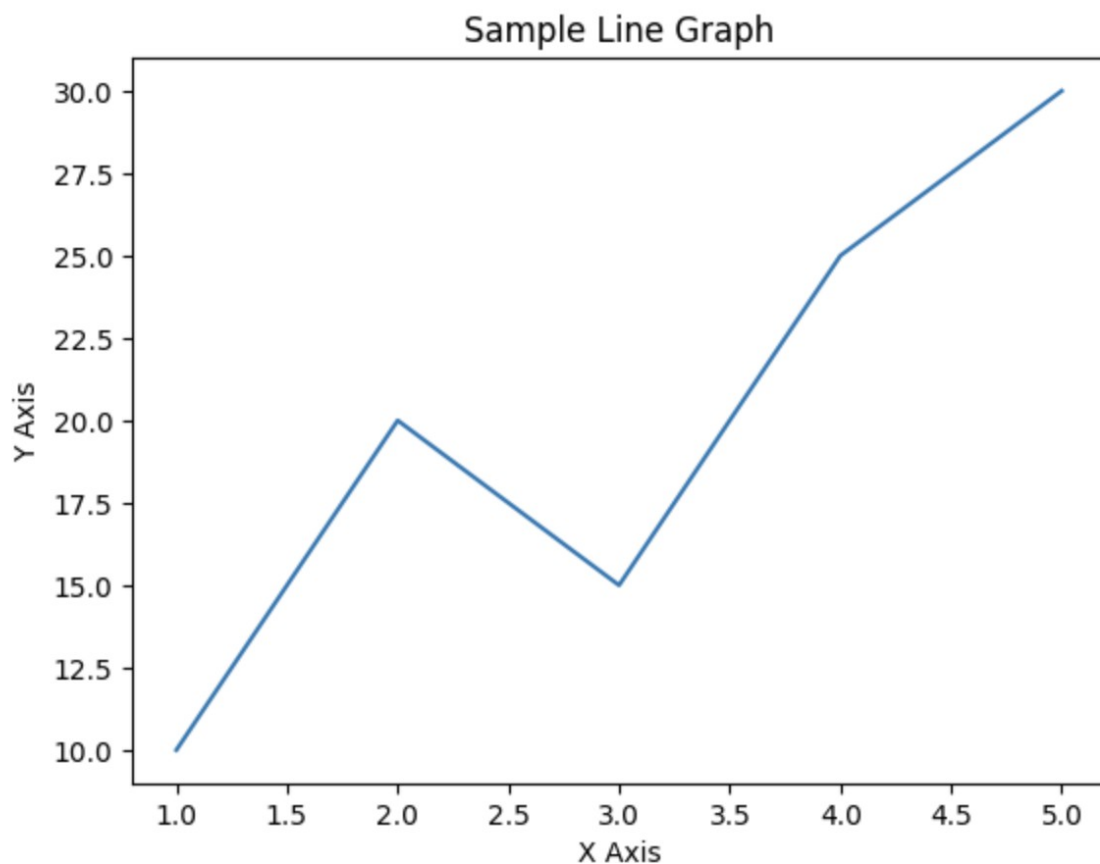
```
df = pd.DataFrame(data)
```

```
print(df)
```

	Name	Marks
0	John	85
1	David	90
2	Alex	95

```
: #To plot a simple line graph
import matplotlib.pyplot as plt
x=[1,2,3,4,5]
y=[10,20,15,25,30]

plt.plot(x,y)
plt.title("Sample Line Graph")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.show()
```



Result:

Thus, Exploratory Data Analysis and data cleaning were performed successfully on the shark attack dataset. Missing values and duplicate records were identified and handled, relevant data types were processed, and important features such as Age, Year, Country, Type, Activity, and Fatal (Y/N) were analyzed for further data analysis.

EX:1b

PERFORMING CRISP-DM AND SEMMA

Aim:

To apply the CRISP-DM (Cross-Industry Standard Process for Data Mining) and SEMMA (Sample, Explore, Modify, Model, Assess) methodologies for systematically understanding, preparing, analyzing, modeling, and evaluating data to extract meaningful insights and support effective decision-making.

Algorithm:

1. Apply the **CRISP-DM** framework by understanding the problem, collecting and preparing the data, building the model, evaluating the results, and presenting the findings.
2. Apply the **SEMMA** framework by sampling the data, exploring its characteristics, modifying and preprocessing it, building the model, and assessing its performance.
3. Compare and interpret the outcomes obtained using both **CRISP-DM** and **SEMMA** methodologies.
4. Draw conclusions based on the analysis and communicate the results for informed decision-making.

Code -

CRISP-DM:

1. TO LOAD THE DATASET

```
+ Code + Markdown | Run All ...
import pandas as pd
data = pd.read_csv("heart.csv")
print(data.head(10))

[3]
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	\
0	52	1	0	125	212	0	1	168	0	1.0	2	
1	53	1	0	140	203	1	0	155	1	3.1	0	
2	70	1	0	145	174	0	1	125	1	2.6	0	
3	61	1	0	148	203	0	1	161	0	0.0	2	
4	62	0	0	138	294	1	1	106	0	1.9	1	
5	58	0	0	100	248	0	0	122	0	1.0	1	
6	58	1	0	114	318	0	2	140	0	4.4	0	
7	55	1	0	160	289	0	0	145	1	0.8	1	
8	46	1	0	120	249	0	0	144	0	0.8	2	
9	54	1	0	122	286	0	0	116	1	3.2	1	

	ca	thal	target
0	2	3	0
1	0	3	0
2	0	3	0
3	1	3	0
4	3	2	0
5	0	2	1
6	3	1	0
7	1	3	0
8	0	3	0
9	2	2	0

2. TO PRINT TAIL AND ANALYSE SHAPE

```
Dataset Shape  
(1025, 14)
```

3. TO PRINT COLUMN TO LIST

```
Columns  
Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',  
       'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'],  
      dtype='object')
```

4. PRINT SAMPLE

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	\
0	52	1	0	125	212	0	1	168	0	1.0	2	
1	53	1	0	140	203	1	0	155	1	3.1	0	
2	70	1	0	145	174	0	1	125	1	2.6	0	
3	61	1	0	148	203	0	1	161	0	0.0	2	
4	62	0	0	138	294	1	1	106	0	1.9	1	
5	58	0	0	100	248	0	0	122	0	1.0	1	

5. TO DESCRIBE THE DATASET

```
Dataset Shape  
(1025, 14)  
  
Columns  
Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',  
       'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'],  
      dtype='object')  
  
...  
target  
1    526  
0    499  
Name: count, dtype: int64
```

6. TO ANALYSE THE MISSING VALUES

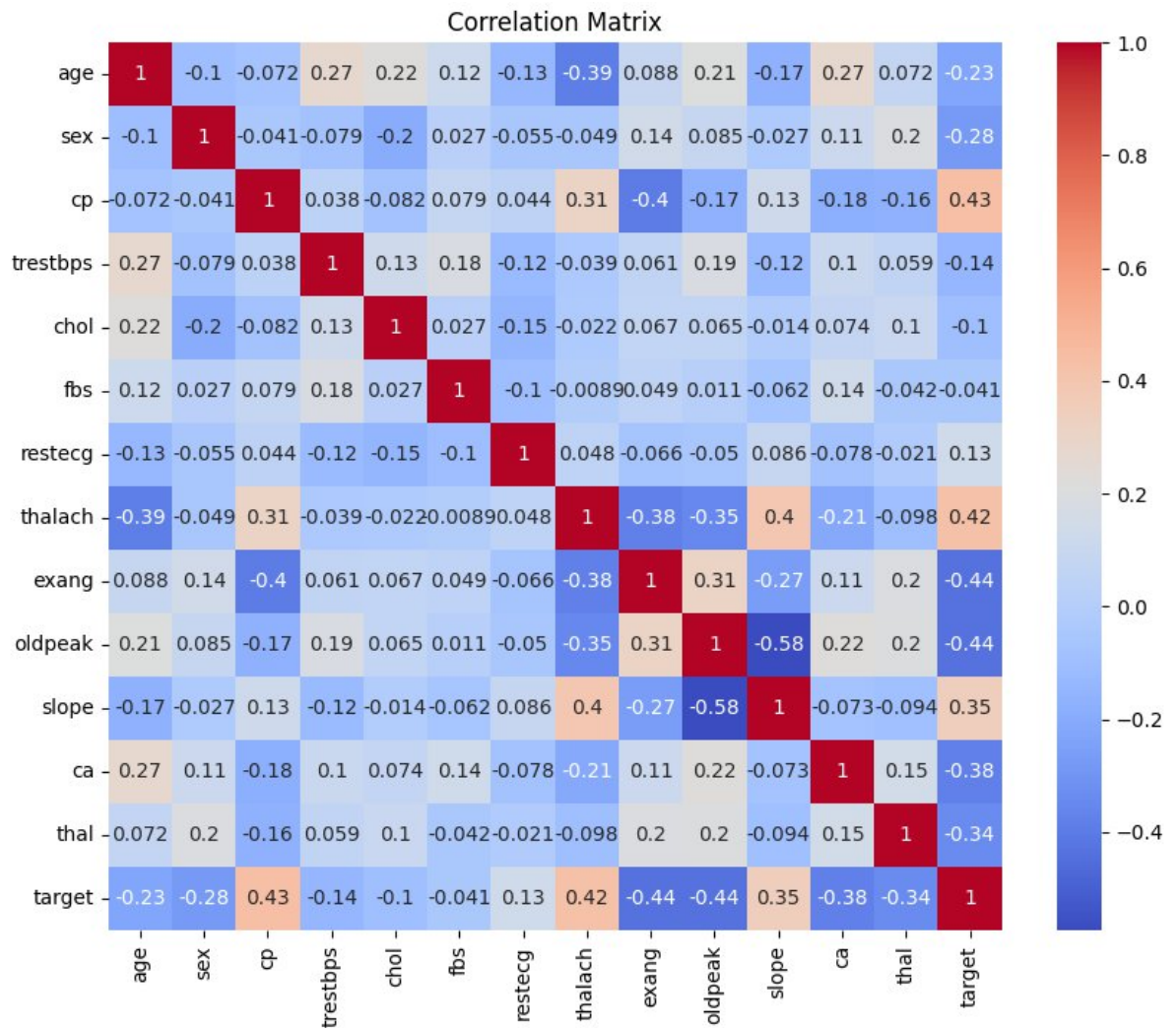
```
Number of null in age is 0
Number of null in sex is 0
Number of null in cp is 0
Number of null in trestbps is 0
Number of null in chol is 0
Number of null in fbs is 0
Number of null in restecg is 0
Number of null in thalach is 0
Number of null in exang is 0
Number of null in oldpeak is 0
Number of null in slope is 0
Number of null in ca is 0
Number of null in thal is 0
Number of null in target is 0
```

7. TO PRINT DUPLICATE ROWS

```
Duplicate Rows:
0

Data Types:
age          int64
sex          int64
cp           int64
trestbps     int64
chol         int64
fbs          int64
restecg      int64
thalach      int64
exang        int64
oldpeak      float64
slope        int64
ca           int64
thal         int64
target       int64
dtype: object
```

8. TO VISUALISE CORRELATION MATRIX



Result:

Thus, the Python environment and Jupyter Notebook were successfully configured and tested. Basic Python programs, NumPy operations, Pandas DataFrame creation, Markdown formatting, and data visualization using Matplotlib were executed successfully, demonstrating the proper functioning of the environment and essential data analysis tools.

EX:2	EDA-DATA IMPORT AND EXPORT
-------------	-----------------------------------

Aim:

To inspect the features of heart disease dataset, detect null values, and export the data into CSV, Excel, and SQLite database formats.

Algorithm:

1. Start the program.
2. Import the required libraries (if needed).
3. Declare and initialize the required variables or data.
4. Perform the required operations or processing.
5. Display the output/results.
6. Stop the program.

Code:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
```

```
# Load dataset
df = pd.read_csv("heart.csv")
```

```
# Display dataset
print(df)
```

```
# Top 2 rows
print("\nTop 2 Rows:")
print(df.head(2))
```

```
# Bottom 2 rows
print("\nBottom 2 Rows:")
print(df.tail(2))
```

```
# Shape
print("\nShape:")
print(df.shape)
```

```
# Info
print("\nInfo:")
df.info()

# Description
print("\nDescription:")
print(df.describe())

# Column names
print("\nColumns:")
print(df.columns)

# Data types
print("\nData Types:")
print(df.dtypes)

# First 5 rows
print("\nFirst 5 Rows:")
print(df.head())

# Missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Fill missing values in cp column (if any)
if "cp" in df.columns:
    df["cp"] = df["cp"].fillna(df["cp"].mean())

# Duplicate rows
print("\nDuplicate Rows:", df.duplicated().sum())

# Remove duplicates
df = df.drop_duplicates()

# Summary statistics
print("\nSummary Statistics:")
print(df.describe())

# Outlier detection using IQR
Q1 = df.quantile(0.25, numeric_only=True)
Q3 = df.quantile(0.75, numeric_only=True)
IQR = Q3 - Q1
```

```

outliers = ((df[Q1.index] < (Q1 - 1.5 * IQR)) |
            (df[Q1.index] > (Q3 + 1.5 * IQR)))

print("\nOutliers Count:")
print(outliers.sum())

# Feature scaling
numerical_columns = ["age", "trestbps", "chol", "thalach", "oldpeak"]

scaler = StandardScaler()
df[numerical_columns] = scaler.fit_transform(df[numerical_columns])

print("\nScaled Data:")
print(df[numerical_columns].head())

# Target distribution
print("\nTarget Distribution:")
print(df["target"].value_counts())

```

Output:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

```
Rows,columns: (1025, 14)
```

```

Dataset info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   age         1025 non-null   int64
 1   sex         1025 non-null   int64
 2   cp          1025 non-null   int64
 3   trestbps    1025 non-null   int64
 4   chol        1025 non-null   int64
 5   fbs         1025 non-null   int64
 6   restecg     1025 non-null   int64
 7   thalach     1025 non-null   int64
 8   exang       1025 non-null   int64
 9   oldpeak     1025 non-null   float64
10   slope       1025 non-null   int64
11   ca          1025 non-null   int64
12   thal        1025 non-null   int64
13   target      1025 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
...

```

```
Dataset columns: Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',  
                        'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'],  
                        dtype='object')
```

```
Number of null in age is 0  
Number of null in sex is 0  
Number of null in cp is 0  
Number of null in trestbps is 0  
Number of null in chol is 0  
Number of null in fbs is 0  
Number of null in restecg is 0  
Number of null in thalach is 0  
Number of null in exang is 0  
Number of null in oldpeak is 0  
Number of null in slope is 0  
Number of null in ca is 0  
Number of null in thal is 0  
Number of null in target is 0
```

```
Duplicate Rows:
```

```
0
```

```
Data Types:
```

```
age          int64  
sex          int64  
cp           int64  
trestbps     int64  
chol         int64  
fbs          int64  
restecg      int64  
thalach      int64  
exang        int64  
oldpeak      float64  
slope        int64  
ca           int64  
thal         int64  
target       int64  
dtype: object
```

```
Summary Statistics:
```

```
count      age      sex      cp      trestbps      chol      fbs  \  
mean      54.42053   0.682119  0.963576  131.602649  246.500000  0.149007  
...  
target  
1        164  
0        138  
Name: count, dtype: int64
```

Result:

Thus, the inspection of features of the customer shopping behavior dataset, detect null values, and export the data into CSV, Excel, and SQLite database formats is done successfully.

EX:3	DATA CLEANING
-------------	----------------------

Aim:

To perform data cleaning using the Pandas library by identifying and handling missing values, duplicate records, and inconsistent data. To prepare a clean and reliable dataset for accurate data analysis and machine learning tasks

Algorithm:

1. Import the Pandas library.
2. Load the dataset into a Pandas Data Frame.
3. Inspect the dataset for missing values, duplicate records, and incorrect data types.
4. Handle missing values by removing or filling them with appropriate values.
5. Remove duplicate records from the dataset.
6. Correct inconsistent or invalid data entries and convert data types if required.
7. Verify the cleaned dataset and save it for further analysis.

Code:

```
import pandas as pd
import numpy as np

# Create DataFrame
df = pd.DataFrame({
    "Name": ["Alice", "Bob", "Alice", "David", "Nyang", "Frank"],
    "Age": [25.0, np.nan, 25.0, 17.0, 30.9, 999.0],
    "Salary": [50000.0, 60000.0, 50000.0, 45000.0, np.nan, 70000.0],
    "Place": ["Chennai", "Delhi", "Chennai", np.nan, "Mumbai", "Delhi"],
    "Joined": ["2021", "2026", "2021", "2019", "2022", "2023"]
})

print("Original DataFrame:")
print(df)

# Drop rows with any null values
print("\nDrop rows with any null values:")
print(df.dropna())
```

```
# Drop rows where all values are null
print("\nDrop rows where all values are null:")
print(df.dropna(how="all"))

# Keep rows with at least 4 non-null values
print("\nRows with at least 4 non-null values:")
print(df.dropna(thresh=4))

# Drop rows where Name or Salary is null
print("\nDrop rows with null in Name or Salary:")
print(df.dropna(subset=["Name", "Salary"]))

# Check duplicates
print("\nDuplicate rows:")
print(df.duplicated())

print("\nNumber of duplicate rows:")
print(df.duplicated().sum())

# Display duplicate rows
print("\nAll duplicate rows:")
print(df[df.duplicated(keep=False)])

# Remove duplicates
df_clean = df.drop_duplicates()

print("\nDataFrame after removing duplicates:")
print(df_clean)

print("\nShape after removing duplicates:")
print(df_clean.shape)

# Remove duplicates based on Name and Age
df.drop_duplicates(
    subset=["Name", "Age"],
    keep="last",
    inplace=True
)

print("\nAfter removing duplicates based on Name and Age:")
print(df)
```

```
# Data types
print("\nData Types:")
print(df.dtypes)

# Fill missing Age values and convert to int
df["Age"] = df["Age"].fillna(0).astype(int)

print("\nAge column after filling nulls:")
print(df["Age"])

print("\nData Types:")
print(df.dtypes)

# Convert Joined to datetime
df["Joined"] = pd.to_datetime(df["Joined"], format="%Y")

print("\nJoined column:")
print(df["Joined"])

print("\nJoined Data Type:")
print(df["Joined"].dtype)

# Convert Age to numeric
df["Age"] = pd.to_numeric(df["Age"], errors="coerce")

print("\nAge column after numeric conversion:")
print(df["Age"])

print("\nData Types:")
print(df.dtypes)

# Rename Name column
df.rename(
    columns={
        "Name": "Full_Name"
    },
    inplace=True
)

print("\nAfter renaming Name:")
print(df)

# Rename remaining columns
```

```

df.rename(
    columns={
        "Age": "Years",
        "Salary": "Income",
        "Place": "City",
        "Joined": "Joining_Date"
    },
    inplace=True
)

print("\nAfter renaming all columns:")
print(df)

# Remove leading/trailing spaces from City
df["City"] = df["City"].str.strip()

print("\nCity values:")
print(df["City"].tolist())

```

Output:

	Name	Age	Salary	Place	Joined
0	Alice	25.0	50000.0	Chennai	2021
1	Bob	NaN	60000.0	Delhi	2026
2	Alice	25.0	50000.0	Chennai	2021
3	David	17.0	45000.0	NaN	2019
4	Nyang	30.9	NaN	Mumbai	2022
5	Frank	999.0	70000.0	Delhi	2023

```
print(df.dropna()) #TO DROP NULL COLUMNS
```

	Name	Age	Salary	Place	Joined
0	Alice	25.0	50000.0	Chennai	2021
2	Alice	25.0	50000.0	Chennai	2021
5	Frank	999.0	70000.0	Delhi	2023

```
print(df.dropna(how="all")) #TO DROP NULL COLUMNS WHEN ALL VALUES ARE NULL
```

	Name	Age	Salary	Place	Joined
0	Alice	25.0	50000.0	Chennai	2021
1	Bob	NaN	60000.0	Delhi	2026
2	Alice	25.0	50000.0	Chennai	2021
3	David	17.0	45000.0	NaN	2019
4	Nyang	30.9	NaN	Mumbai	2022
5	Frank	999.0	70000.0	Delhi	2023

```
print(df.dropna(thresh=4)) #TO DROP NULL ROWS IF THERE ARE MORE THAN 4 ROWS ARE NULL
```

	Name	Age	Salary	Place	Joined
0	Alice	25.0	50000.0	Chennai	2021
1	Bob	NaN	60000.0	Delhi	2026
2	Alice	25.0	50000.0	Chennai	2021
3	David	17.0	45000.0	NaN	2019
4	Nyang	30.9	NaN	Mumbai	2022
5	Frank	999.0	70000.0	Delhi	2023

```
print(df.duplicated()) # TO PRINT THE BOOLEAN VALUES OF DUPLICATED ROWS
```

```
0    False
1    False
2     True
3    False
4    False
5    False
dtype: bool
```

```
print(df.duplicated().sum()) #TO PRINT NUMBER OF ROWS OF DUPLICATED VALUES
```

```
1
```

```
1      0
2     25
3     17
4     30
5    999
Name: Age, dtype: int64
Name      object
Age       int64
Salary    float64
Place     object
Joined    datetime64[ns]
dtype: object
```

```
# Display all duplicate rows in the DataFrame
print(df[df.duplicated(keep=False)])
```

	Name	Age	Salary	Place	Joined
0	Alice	25.0	50000.0	Chennai	2021
2	Alice	25.0	50000.0	Chennai	2021

```
Name      object
Age       float64
Salary    float64
Place     object
Joined    object
dtype: object
```

```
print(df.duplicated())
```

```
0    False
1    False
2     True
3    False
4    False
5    False
dtype: bool
```

```
1    2026-01-01
2    2021-01-01
3    2019-01-01
4    2022-01-01
5    2023-01-01
Name: Joined, dtype: datetime64[ns]
datetime64[ns]
```

	Full_Name	Years	Income	City	Joining_Date
1	Bob	0	60000.0	Delhi	2026-01-01
2	Alice	25	50000.0	Chennai	2021-01-01
3	David	17	45000.0	NaN	2019-01-01
4	Nyang	30	NaN	Mumbai	2022-01-01
5	Frank	999	70000.0	Delhi	2023-01-01

Result:

Thus, the heart disease dataset was successfully cleaned by identifying missing values and applying multiple data-cleaning techniques. The cleaned data is suitable for further analysis.

EX:4

**EXPLORATORY DATA ANALYSIS (EDA) – DATA
INSPECTION, FILTERING, STATISTICAL ANALYSIS
AND CORRELATION**

Aim:

To perform data inspection, filtering, grouping, aggregation, descriptive statistical analysis, and correlation analysis using Pandas and visualize the results using Matplotlib.

Algorithm:

1. Loading and Viewing Data
2. Finding Shape of Dataset
3. Checking Data Types
4. Filtering Data
5. Sorting Data
6. Statistical Analysis
7. Descriptive Statistics
8. Grouping and Aggregation
9. Correlation Analysis
10. Bar Chart
11. Scatter Plot
12. Histogram

Code:

```
import pandas as pd
import matplotlib.pyplot as plt

# Load Dataset
df = pd.read_excel("heart.xlsx")

# 1. First Five Records
print("===== FIRST FIVE RECORDS =====")
print(df.head())

# 2. Shape of Dataset
print("\n===== SHAPE =====")
print(df.shape)

# 3. Data Types
```

```
print("\n===== DATA TYPES =====")
print(df.dtypes)
```

4. Filter Patients with Heart Disease

```
print("\n===== PATIENTS WITH HEART DISEASE =====")
print(df[df["target"] == 1].head())
```

5. Sort by Cholesterol (Descending)

```
print("\n===== SORT BY CHOLESTEROL =====")
print(df.sort_values(by="chol", ascending=False).head())
```

6. Statistical Analysis

```
print("\n===== AGE STATISTICS =====")
print("Mean:", df["age"].mean())
print("Median:", df["age"].median())
print("Mode:", df["age"].mode()[0])
print("Standard Deviation:", df["age"].std())
```

7. Descriptive Statistics

```
print("\n===== DESCRIPTIVE STATISTICS =====")
print(df.describe())
```

8. Grouping and Aggregation

```
print("\n===== AVERAGE CHOLESTEROL BY TARGET =====")
print(df.groupby("target")["chol"].mean())
```

9. Correlation Matrix

```
print("\n===== CORRELATION MATRIX =====")
corr = df.corr(numeric_only=True)
print(corr)
```

10. Bar Chart

```
df.groupby("target")["chol"].mean().plot(kind="bar")
plt.title("Average Cholesterol by Heart Disease")
plt.xlabel("Target (0=No Disease, 1=Disease)")
plt.ylabel("Average Cholesterol")
plt.show()
```

11. Scatter Plot

```
plt.figure(figsize=(8,5))
plt.scatter(df["age"], df["chol"])
plt.title("Age vs Cholesterol")
plt.xlabel("Age")
```

```
plt.ylabel("Cholesterol")
plt.show()
```

12. Histogram

```
plt.figure(figsize=(8,5))
plt.hist(df["age"], bins=20)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

Output:

```
===== FIRST FIVE RECORDS =====
...   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0    52   1   0    125     212   0         1     168      0      1.0     2
1    53   1   0    140     203   1         0     155      1      3.1     0
2    70   1   0    145     174   0         1     125      1      2.6     0
3    61   1   0    148     203   0         1     161      0      0.0     2
4    62   0   0    138     294   1         1     106      0      1.9     1

   ca  thal  target
0   2    3       0
1   0    3       0
2   0    3       0
3   1    3       0
4   3    2       0

===== SHAPE =====
(1025, 14)
```

```
===== DATA TYPES =====
age           int64
sex           int64
cp            int64
trestbps      int64
chol          int64
fbs           int64
restecg       int64
thalach       int64
exang         int64
oldpeak       float64
slope         int64
ca            int64
thal          int64
target        int64
dtype: object
```

===== PATIENTS WITH HEART DISEASE =====

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	\
5	58	0	0	100	248	0	0	122	0	1.0	
10	71	0	0	112	149	0	1	125	0	1.6	
12	34	0	1	118	210	0	1	192	0	0.7	
15	34	0	1	118	210	0	1	192	0	0.7	
16	51	0	2	140	308	0	0	142	0	1.5	

	slope	ca	thal	target
5	1	0	2	1
10	1	0	2	1
12	2	0	2	1
15	2	0	2	1
16	2	1	2	1

===== SORT BY CHOLESTEROL =====

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	\
464	67	0	2	115	564	0	0	160	0	1.6	
158	67	0	2	115	564	0	0	160	0	1.6	
192	67	0	2	115	564	0	0	160	0	1.6	
665	65	0	2	140	417	1	0	157	0	0.8	
123	65	0	2	140	417	1	0	157	0	0.8	

	slope	ca	thal	target
464	1	0	3	1
158	1	0	3	1
192	1	0	3	1
665	2	1	2	1
123	2	1	2	1

===== DESCRIPTIVE STATISTICS =====

	age	sex	cp	trestbps	chol	\
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	
mean	54.434146	0.695610	0.942439	131.611707	246.000000	
std	9.072290	0.460373	1.029641	17.516718	51.59251	
min	29.000000	0.000000	0.000000	94.000000	126.000000	
25%	48.000000	0.000000	0.000000	120.000000	211.000000	
50%	56.000000	1.000000	1.000000	130.000000	240.000000	
75%	61.000000	1.000000	2.000000	140.000000	275.000000	
max	77.000000	1.000000	3.000000	200.000000	564.000000	

	fbs	restecg	thalach	exang	oldpeak	\
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	
mean	0.149268	0.529756	149.114146	0.336585	1.071512	
std	0.356527	0.527878	23.005724	0.472772	1.175053	
min	0.000000	0.000000	71.000000	0.000000	0.000000	
25%	0.000000	0.000000	132.000000	0.000000	0.000000	
50%	0.000000	1.000000	152.000000	0.000000	0.800000	
75%	0.000000	1.000000	166.000000	1.000000	1.800000	
max	1.000000	2.000000	202.000000	1.000000	6.200000	

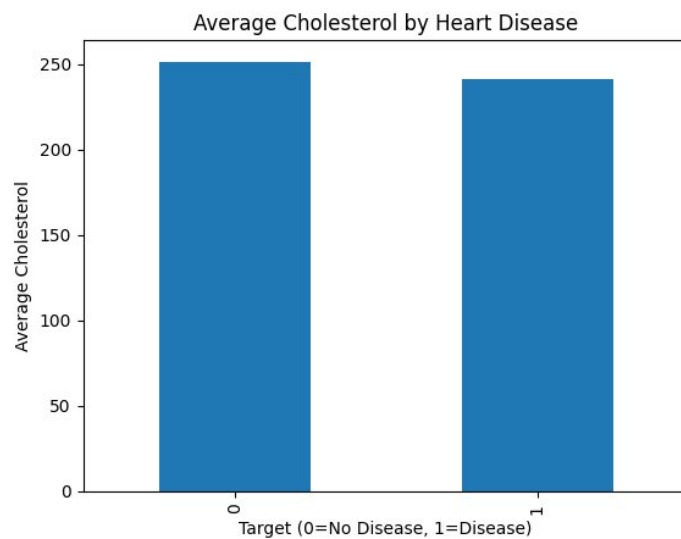
	slope	ca	thal	target
count	1025.000000	1025.000000	1025.000000	1025.000000
mean	1.385366	0.754146	2.323902	0.513171
std	0.617755	1.030798	0.620660	0.500070
min	0.000000	0.000000	0.000000	0.000000
25%	1.000000	0.000000	2.000000	0.000000
50%	1.000000	0.000000	2.000000	1.000000
75%	2.000000	1.000000	3.000000	1.000000
max	2.000000	4.000000	3.000000	1.000000

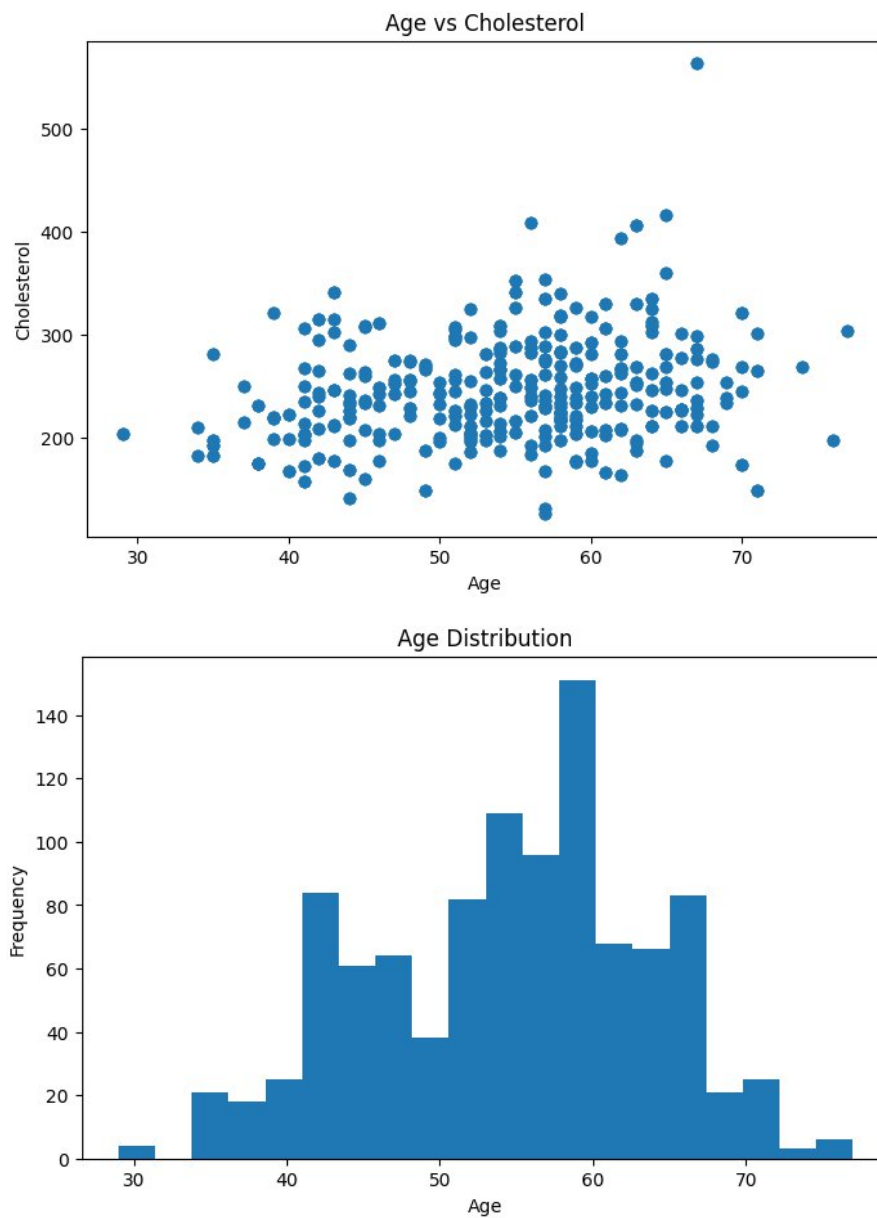
```
===== AGE STATISTICS =====
Mean: 54.43414634146342
Median: 56.0
Mode: 58
Standard Deviation: 9.072290233244281
```

```
===== AVERAGE CHOLESTEROL BY TARGET =====
target
0    251.292585
1    240.979087
Name: chol, dtype: float64
```

```
===== CORRELATION MATRIX =====
          age      sex      cp  trestbps      chol      fbs \
age      1.000000 -0.103240 -0.071966  0.271121  0.219823  0.121243
sex     -0.103240  1.000000 -0.041119 -0.078974 -0.198258  0.027200
cp      -0.071966 -0.041119  1.000000  0.038177 -0.081641  0.079294
trestbps 0.271121 -0.078974  0.038177  1.000000  0.127977  0.181767
chol     0.219823 -0.198258 -0.081641  0.127977  1.000000  0.026917
fbs      0.121243  0.027200  0.079294  0.181767  0.026917  1.000000
restecg -0.132696 -0.055117  0.043581 -0.123794 -0.147410 -0.104051
thalach -0.390227 -0.049365  0.306839 -0.039264 -0.021772 -0.008866
exang    0.088163  0.139157 -0.401513  0.061197  0.067382  0.049261
oldpeak  0.208137  0.084687 -0.174733  0.187434  0.064880  0.010859
slope    -0.169105 -0.026666  0.131633 -0.120445 -0.014248 -0.061902
ca       0.271551  0.111729 -0.176206  0.104554  0.074259  0.137156
thal     0.072297  0.198424 -0.163341  0.059276  0.100244 -0.042177
target   -0.229324 -0.279501  0.434854 -0.138772 -0.099966 -0.041164

          restecg  thalach  exang  oldpeak  slope  ca \
age      -0.132696 -0.390227  0.088163  0.208137 -0.169105  0.271551
sex     -0.055117 -0.049365  0.139157  0.084687 -0.026666  0.111729
cp       0.043581  0.306839 -0.401513 -0.174733  0.131633 -0.176206
trestbps -0.123794 -0.039264  0.061197  0.187434 -0.120445  0.104554
chol     -0.147410 -0.021772  0.067382  0.064880 -0.014248  0.074259
fbs      -0.104051 -0.008866  0.049261  0.010859 -0.061902  0.137156
restecg  1.000000  0.048411 -0.065606 -0.050114  0.086086 -0.078072
thalach  0.048411  1.000000 -0.380281 -0.349796  0.395308 -0.207888
exang    -0.065606 -0.380281  1.000000  0.310844 -0.267335  0.107849
oldpeak  -0.050114 -0.349796  0.310844  1.000000 -0.575189  0.221816
slope     0.086086  0.395308 -0.267335 -0.575189  1.000000 -0.073440
ca       -0.078072 -0.207888  0.107849  0.221816 -0.073440  1.000000
thal     -0.020504 -0.098068  0.197201  0.202672 -0.094090  0.149014
target    0.134468  0.422895 -0.438029 -0.438441  0.345512 -0.382085
```





Result:

Thus, the heart disease dataset was successfully inspected, filtered, sorted, statistically analyzed, and visualized to identify patterns.

EX:5	DATA VISUALIZATION USING MATPLOTLIB
-------------	--

Aim:

To create and analyze various data visualizations such as Line Charts, Bar Charts, Pie Charts, Histograms, Scatter Plots, and Box Plots using Matplotlib for effective data interpretation.

Algorithm:

1. Import the required libraries and load the Heart dataset.
2. Select the required columns for visualization.
3. Prepare or group the data according to the visualization.
4. Choose the appropriate chart type for the selected data.
5. Create the chart using the required Matplotlib function.
6. Add a suitable title, axis labels, and legend if required.
7. Display the visualization and interpret the results.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt

# Load Dataset
df = pd.read_excel("heart.xlsx")

# ===== FIRST FIVE RECORDS =====
print("===== FIRST FIVE RECORDS =====")
print(df.head())

# ===== SHAPE OF DATASET =====
print("\n===== SHAPE OF DATASET =====")
print(df.shape)

# ===== DATA TYPES =====
print("\n===== DATA TYPES =====")
print(df.dtypes)

# ===== DESCRIPTIVE STATISTICS =====
```

```
print("\n===== DESCRIPTIVE STATISTICS =====")
print(df.describe())
```

```
# ===== LINE CHART =====
```

```
age_target = df.groupby("age")["target"].mean()
```

```
plt.figure(figsize=(10,5))
plt.plot(age_target.index, age_target.values)
plt.title("Heart Disease Rate by Age")
plt.xlabel("Age")
plt.ylabel("Average Heart Disease Rate")
plt.show()
```

```
# ===== BAR CHART =====
```

```
sex_target = df.groupby("sex")["target"].mean()
```

```
plt.figure(figsize=(6,4))
plt.bar(sex_target.index.astype(str), sex_target.values)
plt.title("Heart Disease Rate by Gender")
plt.xlabel("Sex (0=Female, 1=Male)")
plt.ylabel("Average Heart Disease Rate")
plt.show()
```

```
# ===== PIE CHART =====
```

```
target_counts = df["target"].value_counts()
```

```
plt.figure(figsize=(7,7))
plt.pie(
    target_counts,
    labels=["Heart Disease", "No Heart Disease"],
    autopct="%1.1f%%"
)
plt.title("Heart Disease Distribution")
plt.show()
```

```
# ===== HISTOGRAM =====
```

```
plt.figure(figsize=(8,5))
plt.hist(df["age"], bins=20)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

```

# ===== SCATTER PLOT =====
plt.figure(figsize=(8,5))
plt.scatter(df["age"], df["chol"])
plt.title("Age vs Cholesterol")
plt.xlabel("Age")
plt.ylabel("Cholesterol")
plt.show()

# ===== BOX PLOT =====
plt.figure(figsize=(6,5))
plt.boxplot(df["chol"])
plt.title("Box Plot of Cholesterol")
plt.ylabel("Cholesterol")
plt.show()

# ===== LINE COMPARISON CHART =====
age_data = df.groupby("age")[["chol", "thalach"]].mean()

plt.figure(figsize=(10,5))
plt.plot(age_data.index, age_data["chol"], label="Cholesterol")
plt.plot(age_data.index, age_data["thalach"], label="Max Heart Rate")
plt.title("Cholesterol vs Max Heart Rate by Age")
plt.xlabel("Age")
plt.ylabel("Average Value")
plt.legend()
plt.show()

```

Output:

```

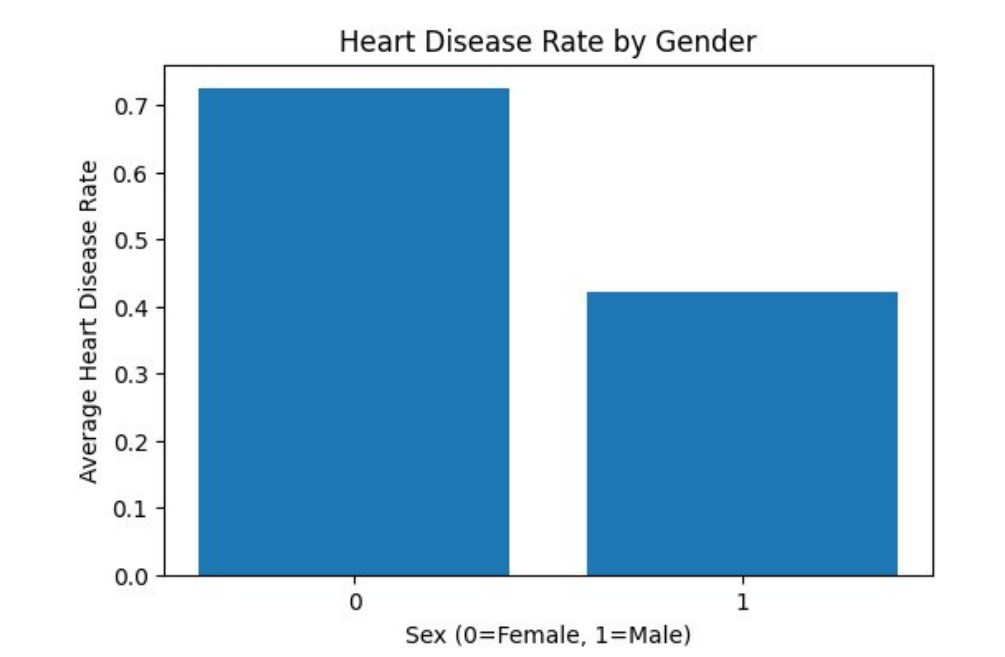
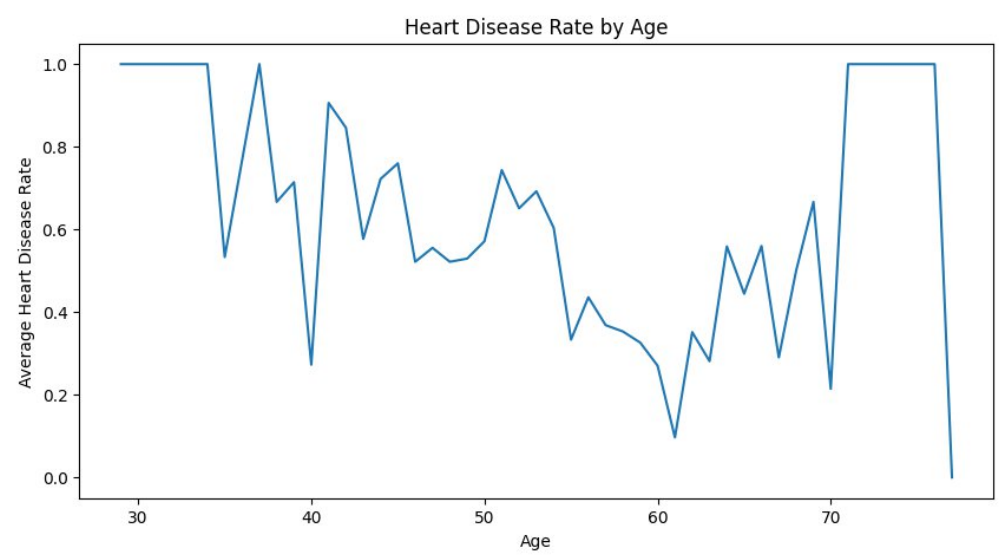
===== SHAPE OF DATASET =====
(1025, 14)

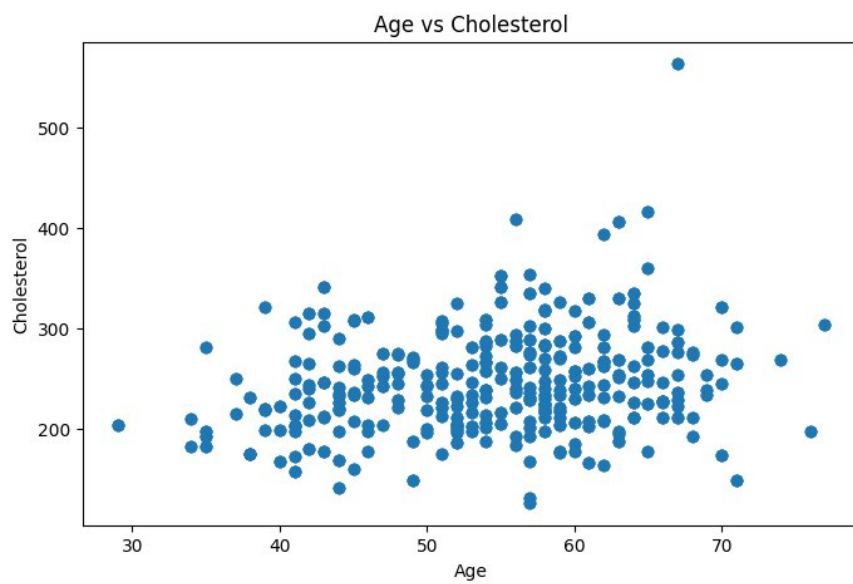
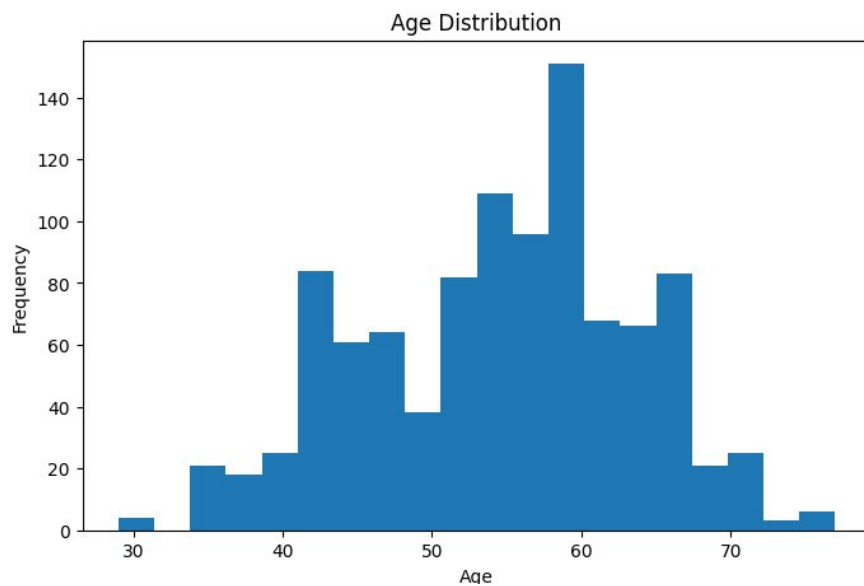
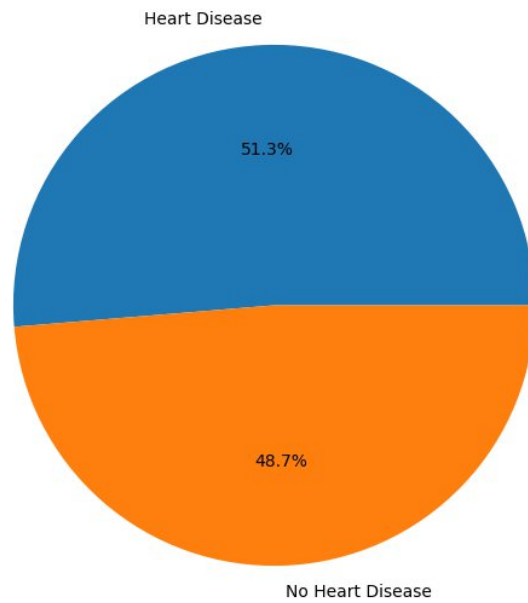
===== DATA TYPES =====
age          int64
sex          int64
cp           int64
trestbps     int64
chol         int64
fbs          int64
restecg      int64
thalach      int64
exang        int64
oldpeak      float64
slope        int64
ca           int64
thal         int64
target       int64
dtype: object

```

===== DESCRIPTIVE STATISTICS =====					
	age	sex	cp	trestbps	chol \
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000
mean	54.434146	0.695610	0.942439	131.611707	246.000000
std	9.072290	0.460373	1.029641	17.516718	51.59251
min	29.000000	0.000000	0.000000	94.000000	126.000000
25%	48.000000	0.000000	0.000000	120.000000	211.000000
50%	56.000000	1.000000	1.000000	130.000000	240.000000
75%	61.000000	1.000000	2.000000	140.000000	275.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000

	fbs	restecg	thalach	exang	oldpeak \
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000
mean	0.149268	0.529756	149.114146	0.336585	1.071512
std	0.356527	0.527878	23.005724	0.472772	1.175053
min	0.000000	0.000000	71.000000	0.000000	0.000000
25%	0.000000	0.000000	132.000000	0.000000	0.000000
50%	0.000000	1.000000	152.000000	0.000000	0.800000
75%	0.000000	1.000000	166.000000	1.000000	1.800000
max	1.000000	2.000000	202.000000	1.000000	6.200000







Result:

The Heart Disease dataset was successfully analyzed and visualized using Matplotlib. The charts highlighted age-wise heart disease trends, gender-based risk patterns, cholesterol distribution, age versus cholesterol relationships, and heart rate variations, providing valuable insights into factors related to heart disease.

EX:6a

DATA VISUALIZATION USING POWER BI

Aim:

To load the heart disease dataset into Power BI, transform the data, and visualize patient health indicators to gain insights into heart disease prediction.

Algorithm:

Step 1: Input Data Loading

- Read the dataset from heart.xlsx.
- Dataset contains the following attributes:
 - age
 - sex
 - cp (chest pain type)
 - trestbps (resting blood pressure)
 - chol (cholesterol)
 - fbs (fasting blood sugar)
 - restecg
 - thalach (maximum heart rate)
 - exang
 - oldpeak
 - slope
 - ca
 - thal
 - target (heart disease: 1 = Yes, 0 = No)

Step 2: Data Cleaning and Transformation

- Check for missing values.
- Verify data types of all attributes.
- Remove duplicate records if present.
- Create calculated columns if required.

Step 3: KPI Calculation

- Calculate Total Patients.
- Calculate Patients with Heart Disease (target = 1).
- Calculate Patients without Heart Disease (target = 0).
- Compute Heart Disease Percentage.

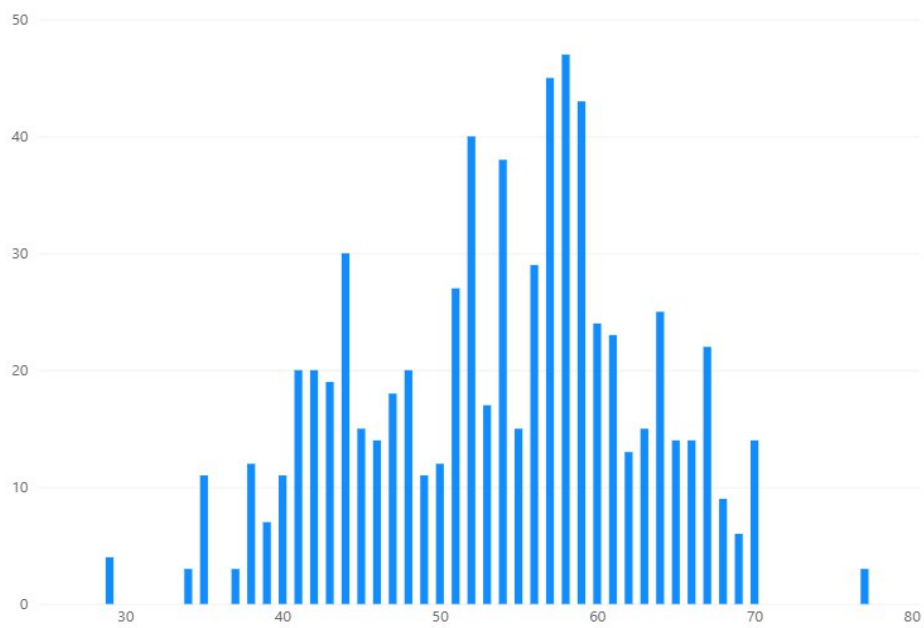
Step 4: Age Group Analysis

- Group patients by age ranges:
 - 30–40
 - 41–50
 - 51–60
 - 61–70

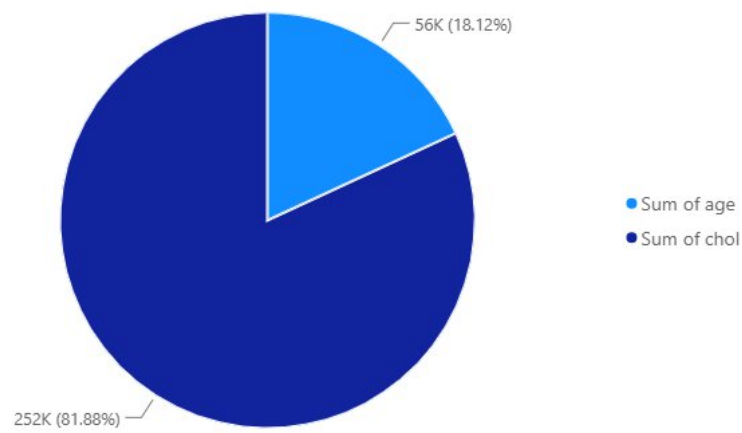
- Above 70
 - Count patients in each age group.
 - Display using a bar chart.
- Step 5: Gender-wise Analysis
- Group data by sex.
 - Calculate:
 - Male Patients
 - Female Patients
 - Heart Disease cases by gender
 - Display using a pie/donut chart.
- Step 6: Cholesterol and Blood Pressure Analysis
- Calculate average cholesterol level.
 - Calculate average resting blood pressure.
 - Compare values between patients with and without heart disease.
 - Display using clustered column charts.
- Step 7: Dashboard Render
- Display KPI Cards:
 - Total Patients
 - Heart Disease Patients
 - Healthy Patients
 - Disease Percentage
 - Display:
 - Age Group Bar Chart
 - Gender Distribution Donut Chart
 - Cholesterol Analysis Chart
 - Blood Pressure Analysis Chart
 - Add slicers for:
 - Age
 - Gender
 - Heart Disease Status
- Step 8: Interactive Filtering
- When a slicer selection changes:
 - Filter the dataset.
 - Recalculate all KPIs and visualizations.
- Update dashboard dynamically.

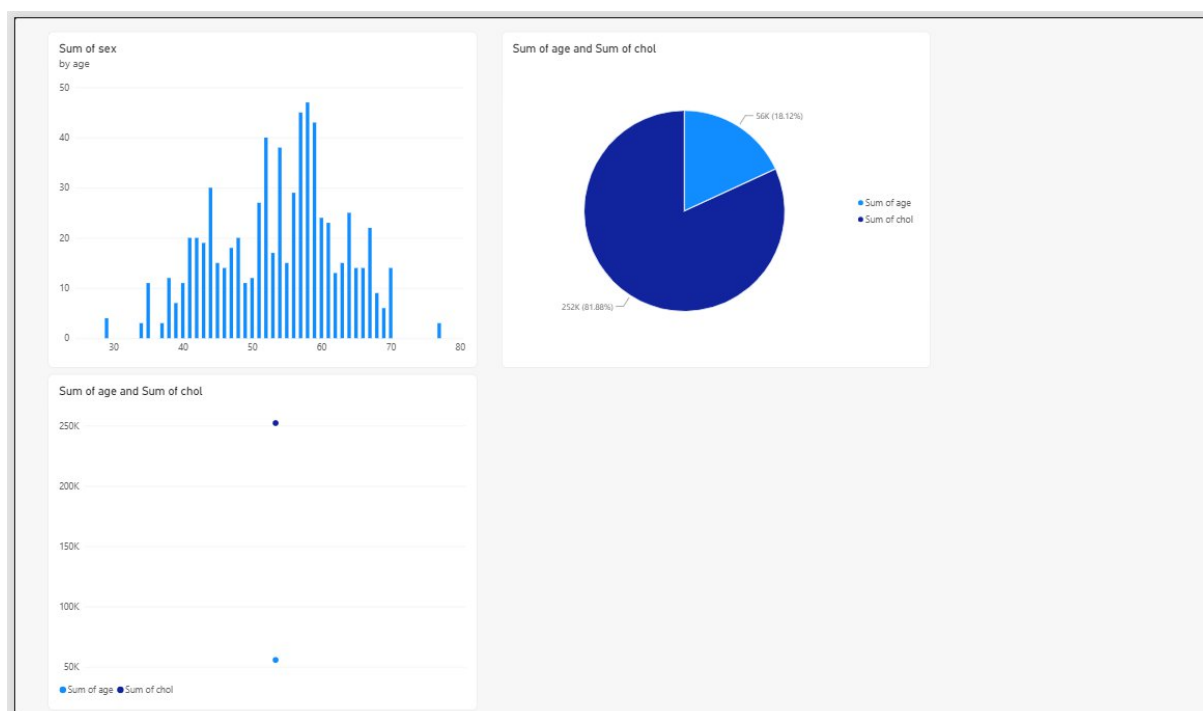
Output:

Sum of sex
by age



Sum of age and Sum of chol





Result:

Thus, the data was successfully loaded, transformed, and visualized using Power BI. The dashboard with KPI cards, bar charts, donut charts, and slicers was created successfully to analyze the data and obtain useful insights.

EX:6b

DATA VISUALIZATION USING POWER BI (ADVANCED SALES ANALYTICS)

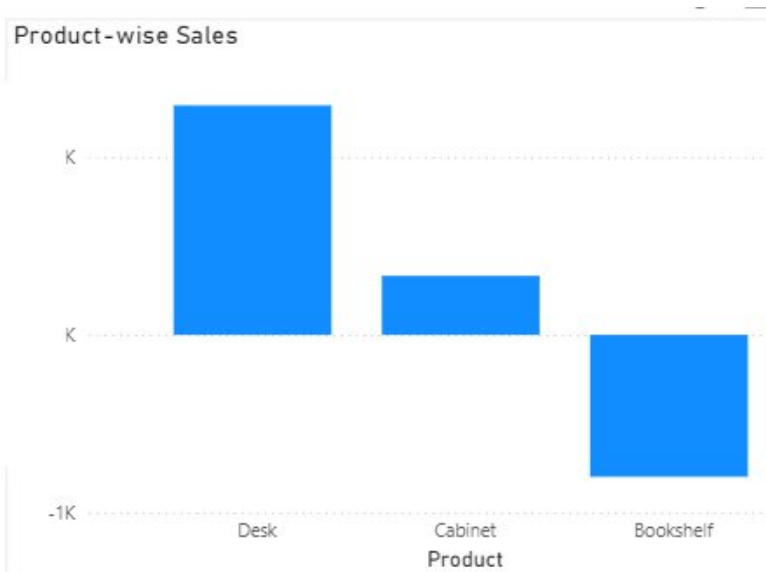
Aim:

To Load the data (Retail_sales) into Power BI and transform the data and visualize it to get insights

Algorithm:

1. Input Data Loading
2. Open **Power BI Desktop**.
3. Import the Retail Sales dataset using **Get Data** → **Excel/CSV**.
4. Load and verify the imported data.
5. Open **Power Query Editor** using **Transform Data**.
6. Clean the dataset by removing duplicates and handling missing values.
7. Rename columns and format data where necessary.
8. Parse and standardize the **Order Date** column.
9. Handle invalid or negative quantity values.
10. Apply changes and load the cleaned dataset.
11. Create a new calculated column **Revenue** using Quantity, Unit Price, and Discount.
12. Build visualizations:
 - a. Line Chart for Revenue by Date
 - b. Bar/Column Chart for Revenue or Quantity by Region
 - c. Donut Chart for Product-wise Quantity Distribution
13. Add slicers for Region and Date filtering.
14. Arrange visuals to create an interactive dashboard.
15. Save the Power BI report (.pbix).
16. Analyse the dashboard to derive sales insights and trends.

Output:



Total Sales

826.63

Total Sales

Average Sales

Average Sales

137.77

Total Orders

Total Orders

6

Region Slicer

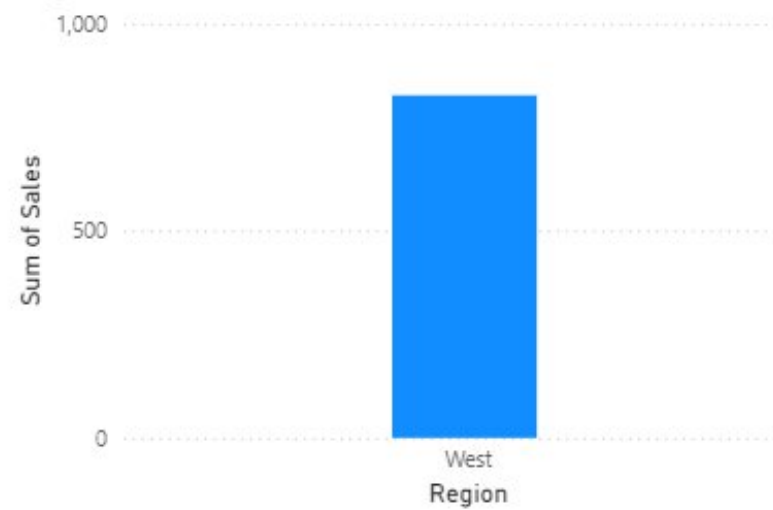
- ☐ East
- ☐ North
- ☐ SOUTH
- ☒ West

Total Quantity

Total Quantity

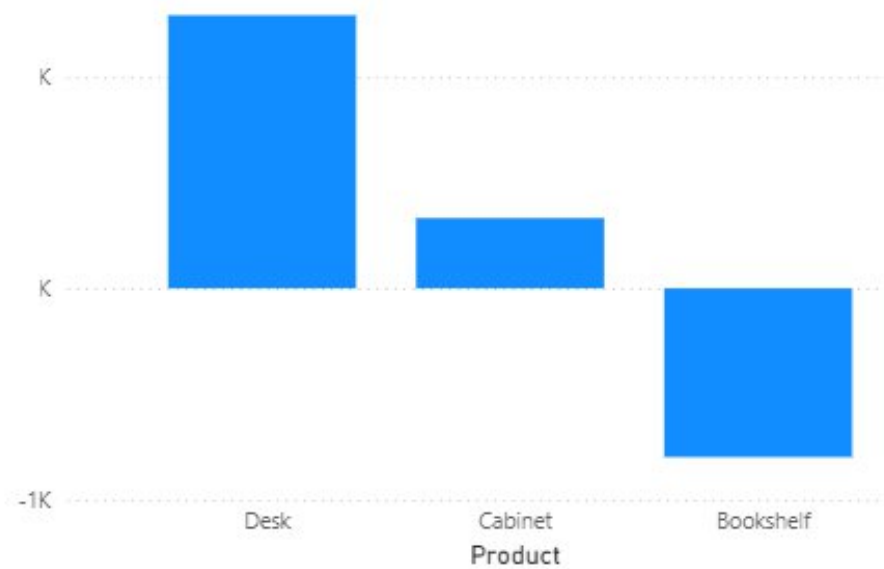
9

Region-wise Sales



Sales by Product Category

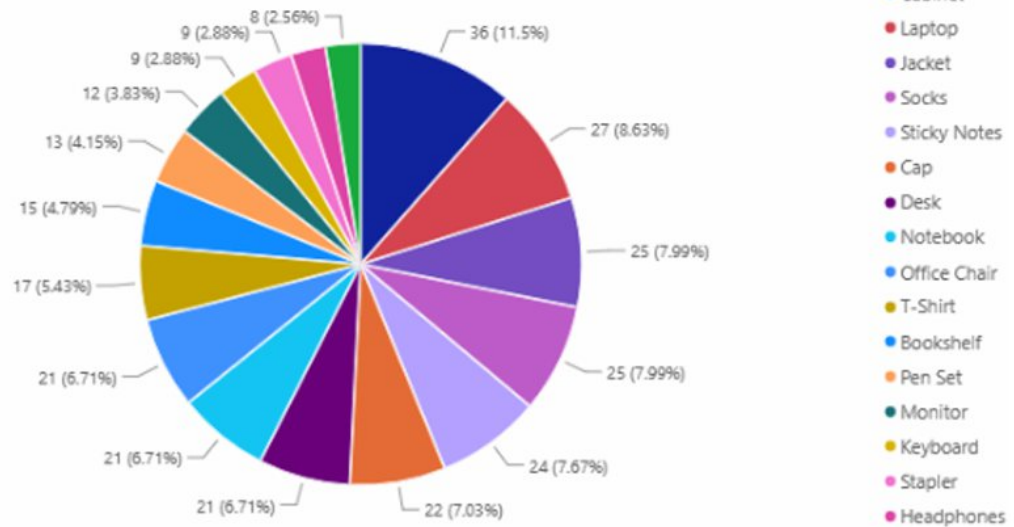
Product-wise Sales



ProductCategory Slicer

- ☐ Clothing
- ☐ Electronics
- ☒ Furniture
- ☐ Stationery

Sum of Quantity
by Product



Result:

Thus, the dataset was successfully imported, cleaned, transformed, and visualized using Power BI. Advanced data transformations, calculated columns, charts, and interactive slicers were created successfully to analyze the dataset and obtain meaningful insights.

EX:6c

CREATE A DASHBOARD IN POWER BI

Aim:

To develop an interactive Heart Disease Dashboard using Microsoft Power BI

Algorithm:

1. Open Microsoft Power BI Desktop.
2. Import the heart.xlsx dataset into Power BI.
3. Examine the available fields such as age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, and target.
4. Select suitable visualizations based on the analysis requirements.
5. Create a KPI Card to display the Total Patients.
6. Create another KPI Card to display the Heart Disease Cases (target = 1).
7. Create a Donut Chart to analyze the distribution of patients with and without heart disease using target.
8. Create a Column Chart to compare heart disease cases across different age groups.
9. Create a Bar Chart to analyze heart disease occurrence by gender (sex).
10. Create a Scatter Chart to study the relationship between cholesterol (chol) and maximum heart rate (thalach).
11. Create a Line Chart to analyze average cholesterol levels across age groups.
12. Create a Table displaying age, sex, cholesterol, maximum heart rate, and target.
13. Create a Slicer for sex.
14. Create another Slicer for target (Heart Disease Status).
15. Arrange and format all visualizations on a single dashboard.
16. Test the slicers and verify that the visualizations respond interactively.
17. Analyze the completed dashboard and derive meaningful insights regarding heart disease risk factors

Output:

Diagram 1: KPI Card – Age

Procedure:

Open the Power BI report.

1. Select the Card visual from the Visualizations pane.
2. Drag the **age** field from the Data pane.
3. Drop it into the Value field of the Card.
4. Power BI automatically performs the aggregation, such as Sum of age.
5. Rename the card title as Total Followers.
6. Format the card for better visibility.

Output:



Diagram 2: KPI Card – Age

Procedure

1. Select a new Card visual.
2. Locate the **age** field in the Data pane.
3. Drag **age** into the Value field.
4. Verify that Power BI calculates the required aggregation.
5. Rename the card as age.
6. Format the card appropriately.

Output

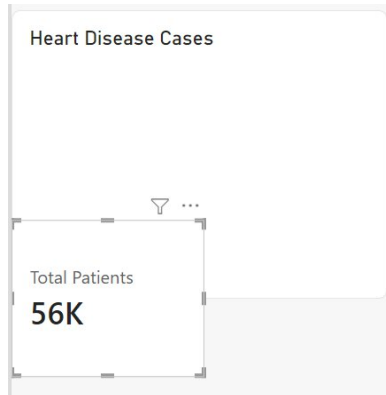


Diagram 3: Line Chart – Age Group

Procedure

1. Select the Line Chart visualization.
2. Drag Age Group into the X-axis.
3. Drag chol (Cholesterol) into the Y-axis.
4. Change the aggregation of chol to Average.
5. Power BI plots the average cholesterol levels across different age groups.
6. Add a suitable title such as Average Cholesterol by Age Group.
7. Format the chart for clear visualization by adjusting labels, title, and axis settings.

Output

Average of chol
by age

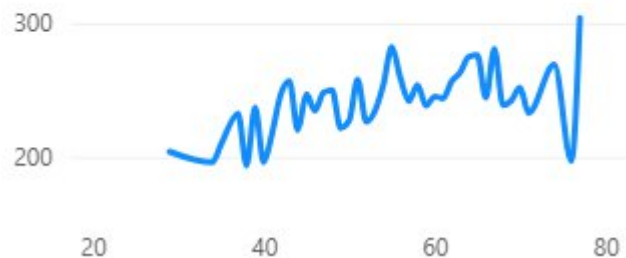


Diagram 4: Column Chart – Heart Disease Cases by Age Group

Procedure

1. Select the Clustered Column Chart visualization.
2. Drag Age Group into the X-axis.
3. Drag target into the Y-axis.
4. Change the aggregation of target to Sum.
5. Power BI calculates the number of heart disease cases for each age group.
6. Add the title Heart Disease Cases by Age Group.
7. Format the chart for better readability by adjusting colors, labels, and axis titles.

Output

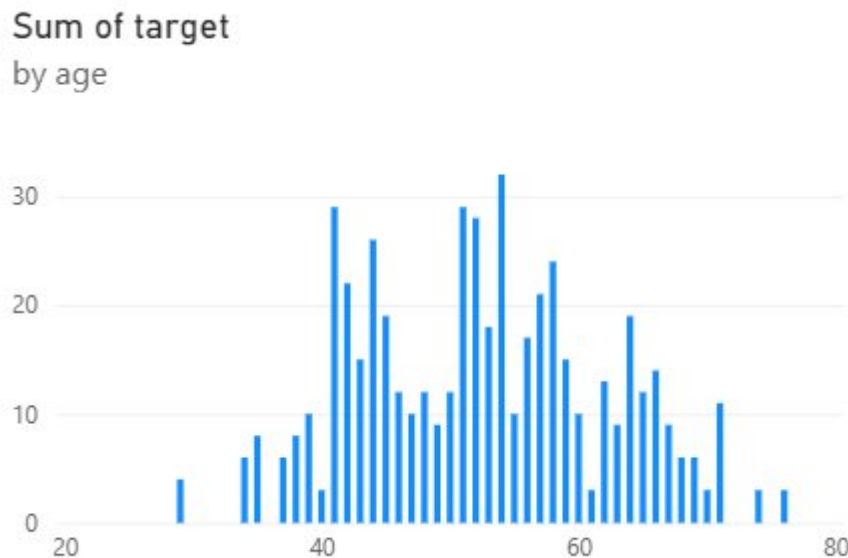


Diagram 5: Donut/Pie Chart – Heart Disease Distribution

Procedure

1. Select the Donut Chart visual.
2. Drag target into the Legend field.
3. Drag target into the Values field.
4. Change the aggregation to Count if required.
5. Power BI creates separate sections for patients with heart disease (1) and without heart disease (0).
6. Add the title Heart Disease Distribution.
7. Format the chart and enable Data Labels to display counts or percentages.

Output

Count of target
by target

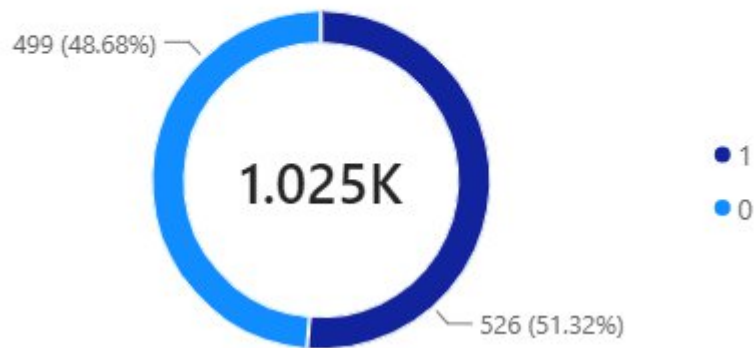


Diagram 6: Table – Top Posts by Engagement

Procedure

1. Select the Table visualization.
2. Drag age into the table.
3. Drag sex into the table.
4. Drag chol (Cholesterol) into the table.
5. Drag thalach (Maximum Heart Rate) into the table.
6. Drag target into the table.
7. Sort the chol column in descending order (or sort target to view heart disease cases first).
8. The patient records with higher cholesterol levels or heart disease status appear at the top.

Output

age	sex	chol	thalach	target
29	1	204	202	1
34	0	210	192	1
34	1	182	174	1
35	0	183	182	1
35	1	192	174	1
35	1	198	130	0
35	1	282	156	0
37	0	215	170	1
37	1	250	187	1
38	1	175	173	1

Diagram 7: Gender Slicer Procedure

Procedure :

1. Select the Slicer visual.
2. Choose List Slicer.
3. Drag sex into the Field area.
4. The available gender categories are displayed as a list.
5. Select a gender to filter the dashboard visuals accordingly.
6. Clear the selection to return to the complete dataset.

Output



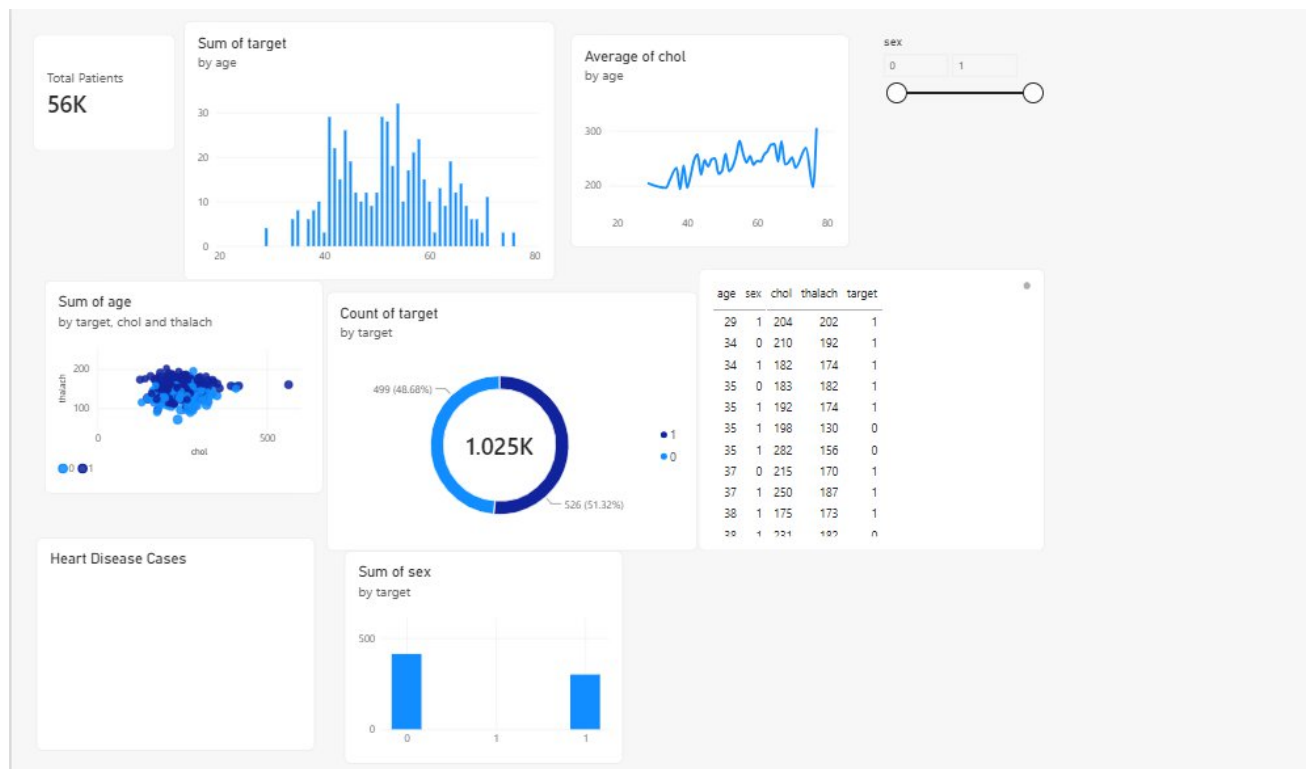
FINAL DASHBOARD

Procedure

1. Arrange the Total Patients and Heart Disease Cases KPI cards at the top of the dashboard.
2. Place the Heart Disease Cases by Age Group column chart in the main analysis area.
3. Place the Average Cholesterol by Age Group line chart beside or below it.
4. Place the Heart Disease Distribution donut chart in the dashboard.
5. Add the Patient Details table.
6. Place the Gender and Heart Disease Status slicers in an easily accessible area.
7. Add the dashboard title:

HEART DISEASE ANALYSIS DASHBOARD

8. Resize and align all visualizations properly.
9. Apply appropriate formatting, colors, and titles to improve readability.
10. Test the slicers to verify that the dashboard responds dynamically and filters all visuals accordingly.



Result:

Thus, an interactive Heart Disease Dataset was successfully created using Microsoft Power BI.

EX:7a

CREATE DASHBOARD USING TABLEAU WITH STUDENT MARKSHEET DATASET

Aim:

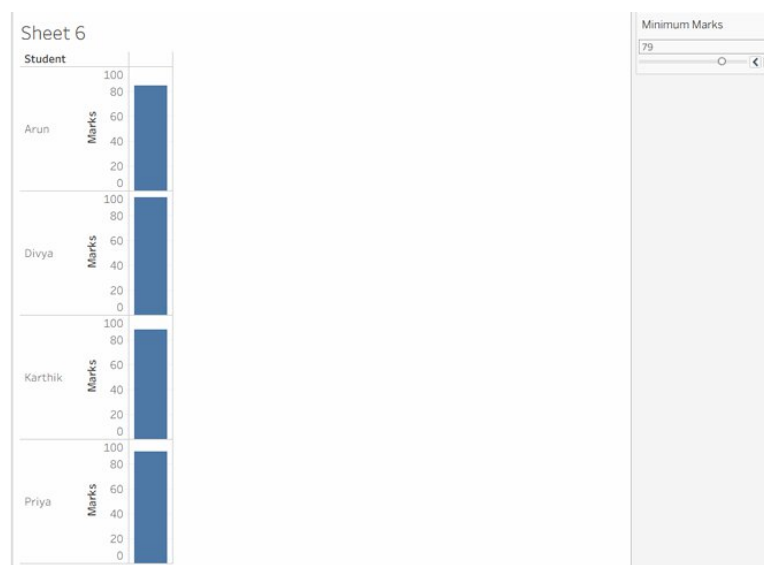
To create an interactive Tableau dashboard to visualize and analyze students' marks across different departments using bar charts, line charts, a pie chart, and filters.

Algorithm:

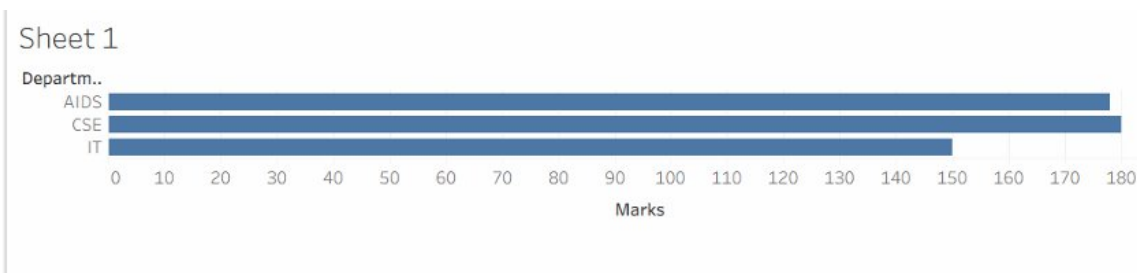
1. Import the student marks dataset into Tableau.
2. Create individual worksheets for different visualizations.
3. Create a bar chart to compare marks by department.
4. Create line charts to display marks for each student.
5. Create a table showing department, student, and marks.
6. Create a pie chart to represent marks by department.
7. Add department and mark filters to the dashboard.
8. Arrange all worksheets into a single dashboard.
9. Apply suitable titles, labels, and formatting.
10. Verify the dashboard and its interactive filters.

Output:

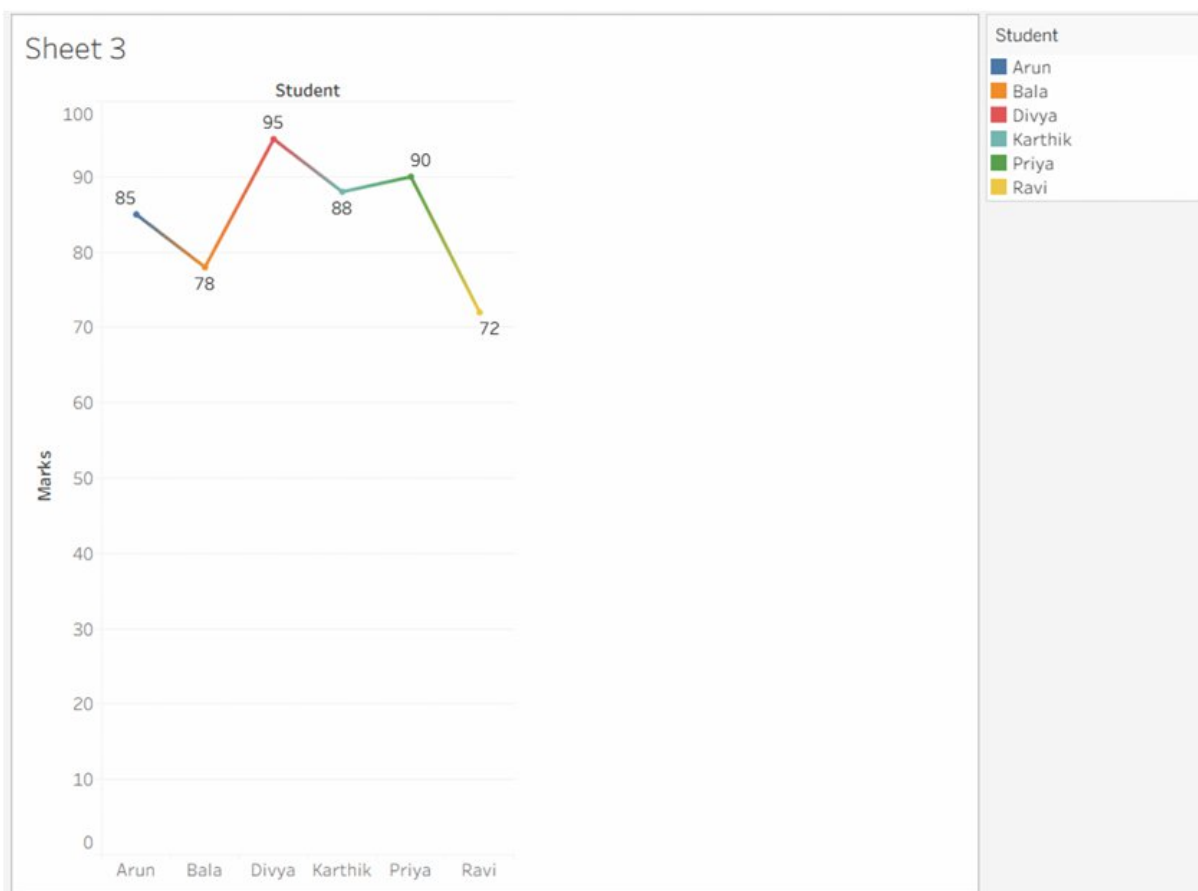
1. Open Tableau and connect to the student marks dataset.
2. Drag the required fields such as Department, Student, and Marks into the respective shelves.



3. Create the department-wise bar chart using Department and Marks.



4. Create line charts using Student and Marks, and enable mark labels.

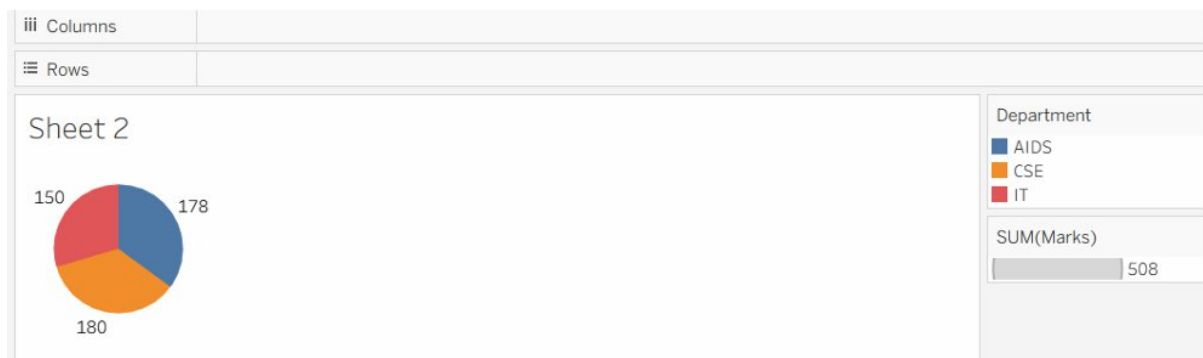


5. Create a table containing Department, Student, and Marks.

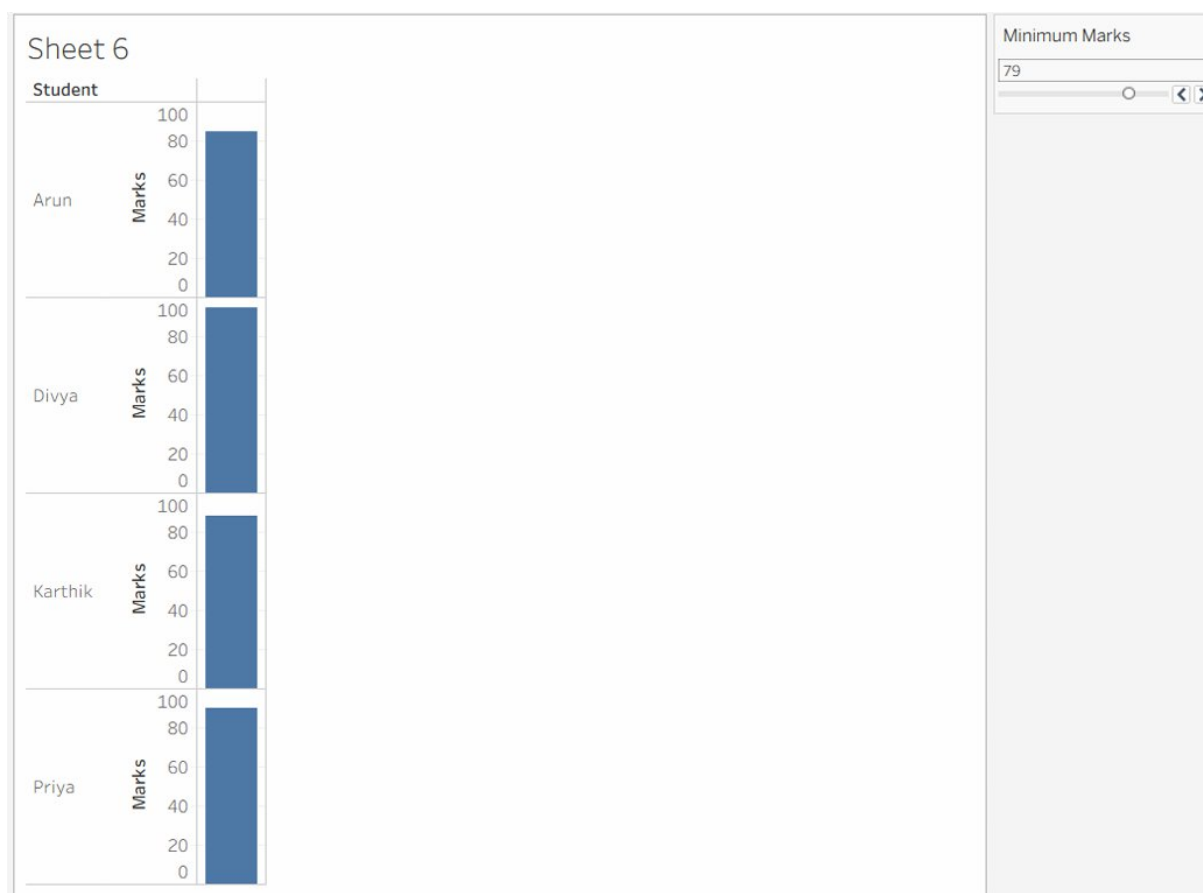
Sheet 4

Departm..	Student	Marks	Percenta..
AIDS	Karthik	88.00	88.00%
	Priya	90.00	90.00%
CSE	Arun	85.00	85.00%
	Divya	95.00	95.00%
IT	Bala	78.00	78.00%
	Ravi	72.00	72.00%

6. Select the Pie Chart option from Show Me and use Department and Marks.



7. Create a new Dashboard and drag the created worksheets into it.



Result:

The Tableau dashboard was successfully created to analyze student performance. It displays department-wise marks, student-wise performance, tabular details, and department-wise distribution using multiple visualizations. Interactive filters allow the user to view the required departments and students based on minimum marks.

EX:7a

CREATE DASHBOARD USING TABLEAU

Aim:

To develop an interactive **Heart Disease Analytics Dashboard** using Tableau by analyzing heart disease patient data through KPI cards, charts, tables, and interactive filters, in order to understand patient demographics, heart disease prevalence, age-group distribution, gender-wise analysis, risk factors, and overall disease patterns for effective data-driven healthcare insights.

Algorithm:

1. Open Tableau Desktop.
2. Connect the Heart Disease Dataset.
3. Create KPI cards for:
 4. Total Patients
 5. Heart Disease Cases
 6. Heart Disease Percentage
 7. Average Age
8. Create a Column Chart for Heart Disease by Age Group.
9. Create a Donut Chart for Heart Disease by Gender.
10. Create a Bar Chart for Heart Disease by Chest Pain Type.
11. Create a Scatter Plot for Cholesterol vs Maximum Heart Rate.
12. Create a Table showing patient details and disease status.
13. Add filters for Age Group, Gender, and Heart Disease Status.
14. Create a new Dashboard and add all visualizations.
15. Arrange KPI cards at the top and charts below.
16. Test the filters and verify interactive analysis.

Output:

Diagram 1: Total Followers KPI

Procedure

1. Create a new worksheet in Tableau.
2. Select the Patient ID field (or any unique patient identifier) from the Data pane.

3. Drag Patient ID to the Text option in the Marks card.
4. Change the aggregation to COUNT to calculate the total number of patients.
5. Format the displayed number to make it clearly visible.
6. Rename the worksheet as Total Patients.

Output

The Total Followers KPI displays:

Total Patients

55,795

Diagram 2: Heart Disease Cases

KPI Procedure

1. Create a new worksheet.
2. Select the Target field from the Data pane.
3. Drag Target to the Text option in the Marks card.
4. Tableau calculates the SUM(Target), representing the total number of heart disease cases.
5. Format the number for better visibility.
6. Rename the worksheet as Heart Disease Cases.

Output

The KPI displays the total aggregated engagement rate:

Total Rate

1,025

Diagram 3:

Heart Disease by Age Group Procedure

1. Create a new worksheet.
2. Drag Age Group to the Columns shelf.
3. Drag Target to the Rows shelf.
4. Ensure the aggregation is SUM(Target).

5. Select Bar Chart from Show Me if required.
6. Tableau generates a chart showing heart disease cases across different age groups.
7. Format the chart and provide a suitable title.
8. Rename the worksheet as Heart Disease Cases by Age Group.

Output

A bar chart is generated showing the distribution of heart disease cases among different age groups.

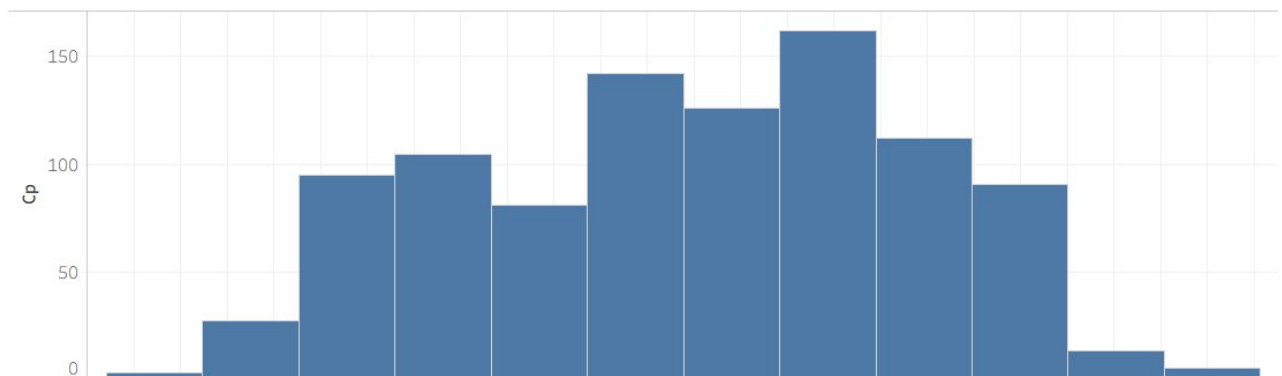


Diagram 4: Heart Disease by Chest Pain Type

Procedure

1. Create a new worksheet.
2. Drag Chest Pain Type (cp) to the Columns shelf.
3. Drag Heart Disease Cases (SUM(Target)) to the Rows shelf.
4. Select the Bar/Column Chart from Show Me.
5. Tableau generates a separate bar for each chest pain type.
6. Add the title heart disease by Chest Pain Type.
7. Format the chart for easy comparison.

Output:

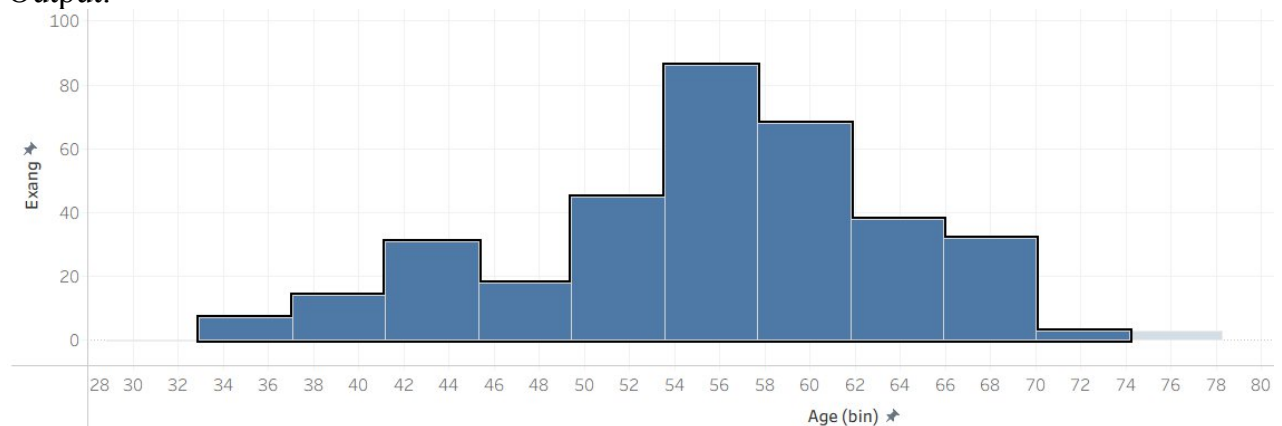


Diagram 5: Heart Disease by

Gender Procedure

1. Create a new worksheet.
2. Select Pie Chart from Show Me.
3. Drag Gender (Sex) to Color in the Marks card.
4. Drag Heart Disease Cases (SUM(Target)) to Angle.
5. Tableau divides the pie into sections according to gender.
6. Add the title heart disease by Gender.
7. Format the visualization appropriately.

Output:

Sheet 2

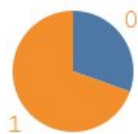


Diagram 6: Age Group Filter

Procedure

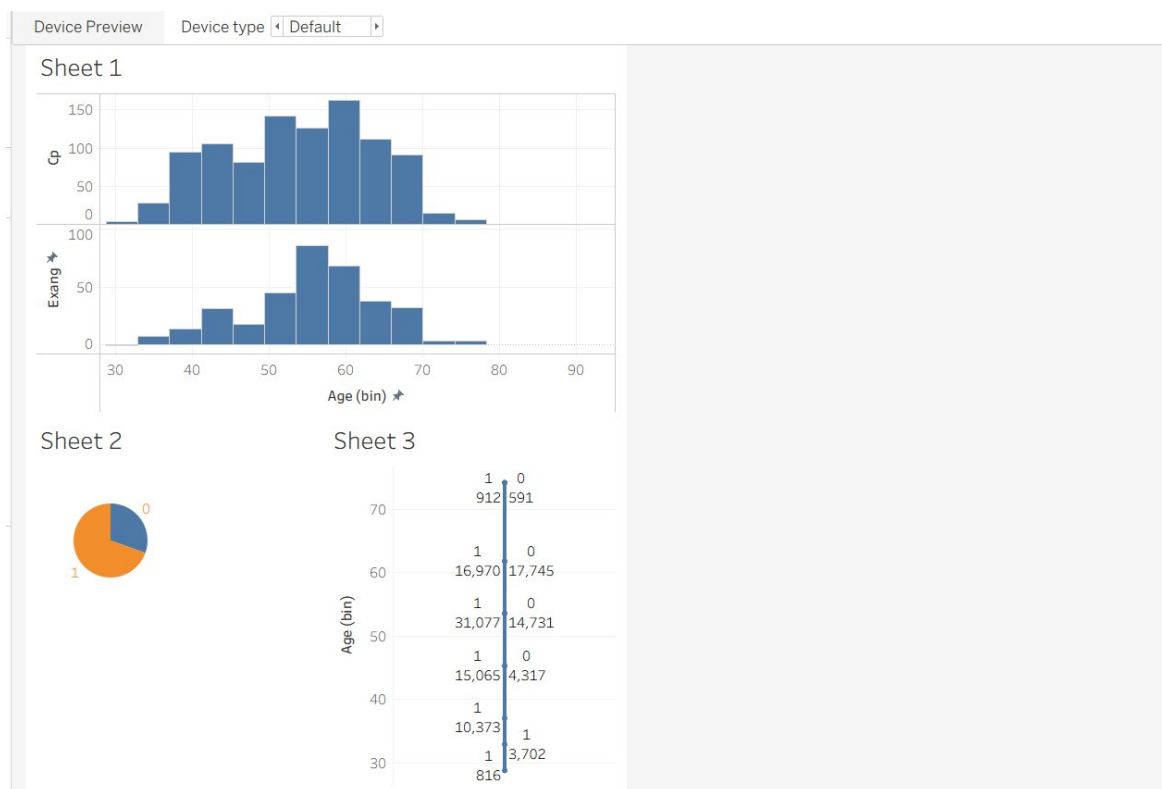
1. Add Age Group to the Filters shelf.
2. Select All Age Groups.
3. Click OK.
4. Select Show Filter.
5. The age groups are displayed as an interactive filter.
6. Select any age group to analyse that group separately.

Output:

Procedure

1. Select Dashboard → New Dashboard.
2. Tableau displays the available worksheets on the left side.
3. Drag the required worksheets onto the dashboard.
4. Add the following visualizations:

- Total Patients
 - Heart Disease Cases
 - Heart Disease Cases by Age Group
 - Heart Disease by Gender
 - Heart Disease by Chest Pain Type
 - Top Patients Table
5. Arrange the KPI cards at the top.
 6. Place the charts below the KPI cards.
 7. Place the patient details table at the bottom.
 8. Add Age Group, Gender, and Heart Disease Status filters.
 9. Add the title:
 10. HEART DISEASE ANALYTICS DASHBOARD
 11. Resize and align the visualizations.
 12. Test the filters by selecting different age groups, genders, and disease statuses.
 13. Verify that the dashboard updates according to the selected filters.



RESULT:

Thus, the **Heart Disease Analytics Dashboard** was successfully created using Tableau. The dashboard provides an interactive analysis of heart disease data through KPI cards,

charts, tables, and filters, helping users understand disease prevalence, patient demographics, and key health indicators.

Ex.no:8	MINI PROJECT – SOCIAL MEDIA ANALYTICS

Domain and Problem Statement

Domain: Social Media Analytics & Instagram Performance Analysis

Problem Statement:

Businesses and content creators generate large volumes of Instagram data but often lack clear insights into content performance, audience engagement, reach, and follower growth. This project addresses these challenges by transforming raw Instagram post data into structured business intelligence using Power BI and Tableau, enabling analysis of engagement trends, content performance, posting patterns, traffic sources, and overall Instagram performance to support data-driven content and marketing decisions.

Dataset Description

Dataset Name: Instagram Analytics Dataset

Granularity: Each row represents a single Instagram post.

Timeframe Covered: 2025

Key Entities & Attributes:

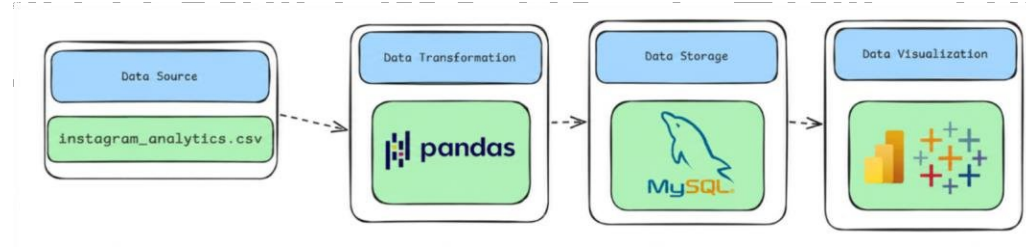
- **Account & Content:** account_id, account_type, media_type, content_category, caption_length, hashtags_count, has_call_to_action.
- **Post Details:** post_id, post_date, post_datetime, post_hour, day_of_week.
- **Engagement Metrics:** likes, comments, shares, saves, engagement_rate.
- **Reach & Growth:** reach, impressions, followers_gained, traffic_source.
- **Performance:** performance_bucket_label.

Data Source:

Primary Source: Instagram Analytics Dataset prepared for social media performance analysis.

Storage Infrastructure: Cleaned and pre-processed Instagram post records maintained in an Excel dataset and imported into Microsoft Power BI and Tableau for interactive dashboard development and analysis.

Architecture:



Program:

1. Python – Data Cleaning &

Transformation import pandas as

pd df =

```
pd.read_csv("instagram_analytics.csv")
```

```
# Create a copy for processing instagram = df.copy() # Handle
```

```
missing values instagram["account_type"] =
```

```
instagram["account_type"].fillna("Unknown")
```

```
instagram["media_type"] =
```

```
instagram["media_type"].fillna("Unknown")
```

```
instagram["content_category"] =
```

```
instagram["content_category"].fillna("Unknown")
```

```
instagram["traffic_source"] =
```

```
instagram["traffic_source"].fillna("Unknown")
```

```
# Replace missing numerical values with 0
```

```
numeric_columns = [
```

```
    "likes",  
    "comments",  
    "shares",  
    "saves",  
    "reach",  
    "impressions",  
    "followers_gained"
```

```
]
```

```
for column in numeric_columns:
```

```
    instagram[column] = instagram[column].fillna(0)
```

```

# Remove duplicate records

instagram.drop_duplicates(inplace=True)

# Convert date column
instagram["post_date"] = pd.to_datetime(
instagram["post_date"], errors="coerce"
)
instagram.columns =
( instagram.columns
    .str.lower()
    .str.strip()
    .str.replace(" ", "_")
)
# Display dataset information
print("Original Shape:",
df.shape) print("Cleaned
Shape:", instagram.shape)

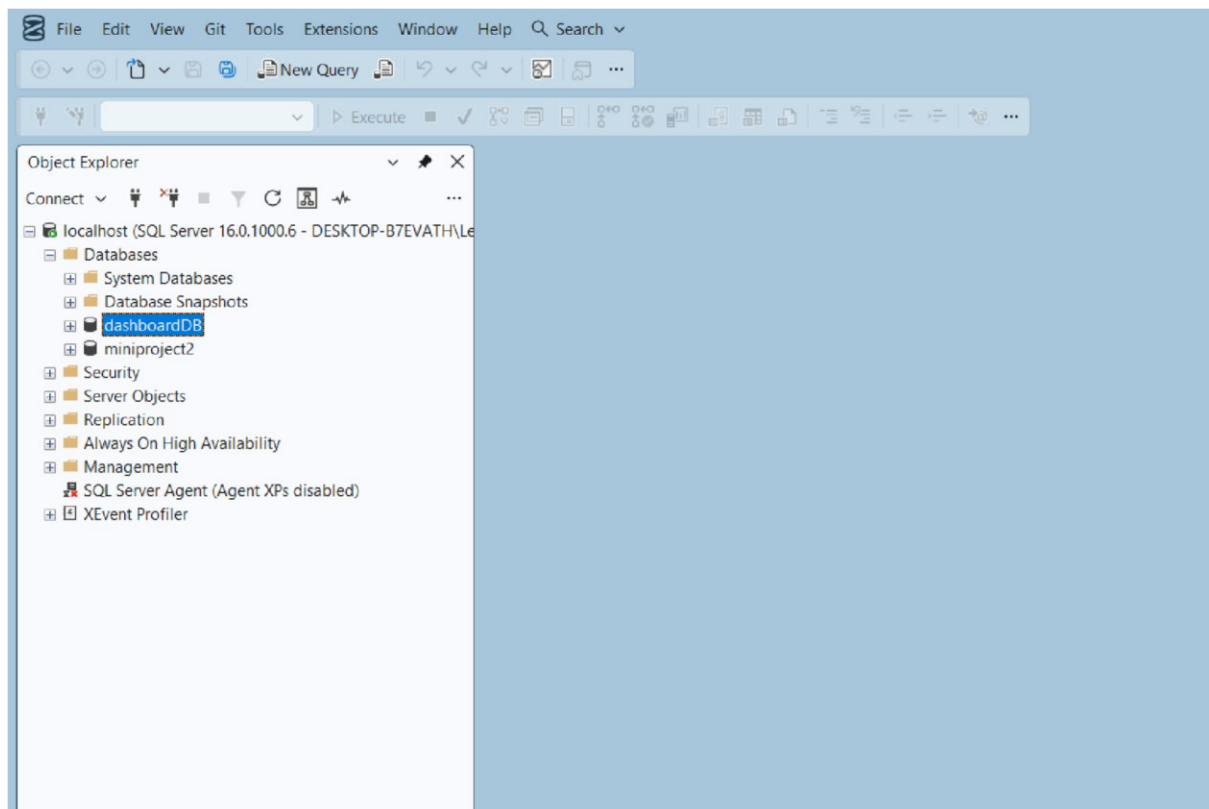
print("\nDataset Preview:")
print(instagram.head())

print("\nDataset Information:")
print(instagram.info())

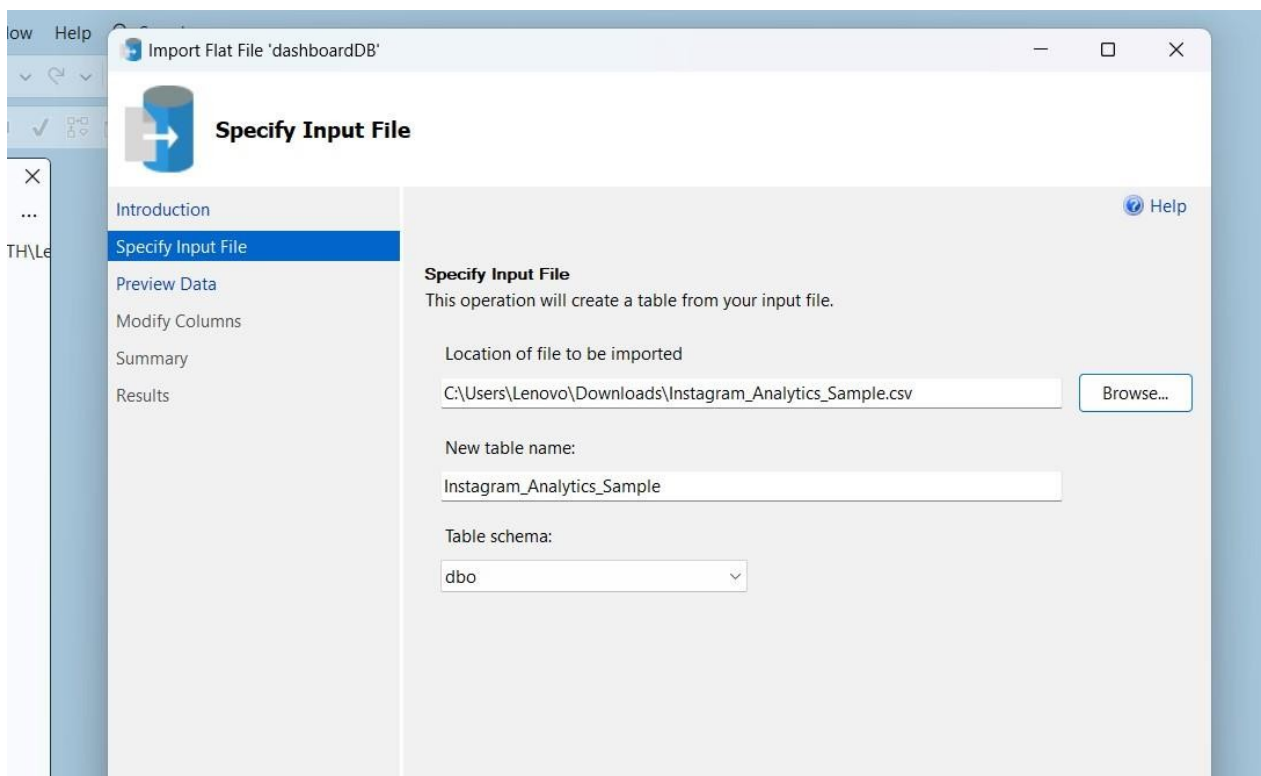
```

2. SQL SERVER CONNECTIVITY

INSTALL SQL SERVER MANAGEMENT STUDIO 1) Create database named “dashboardDB”

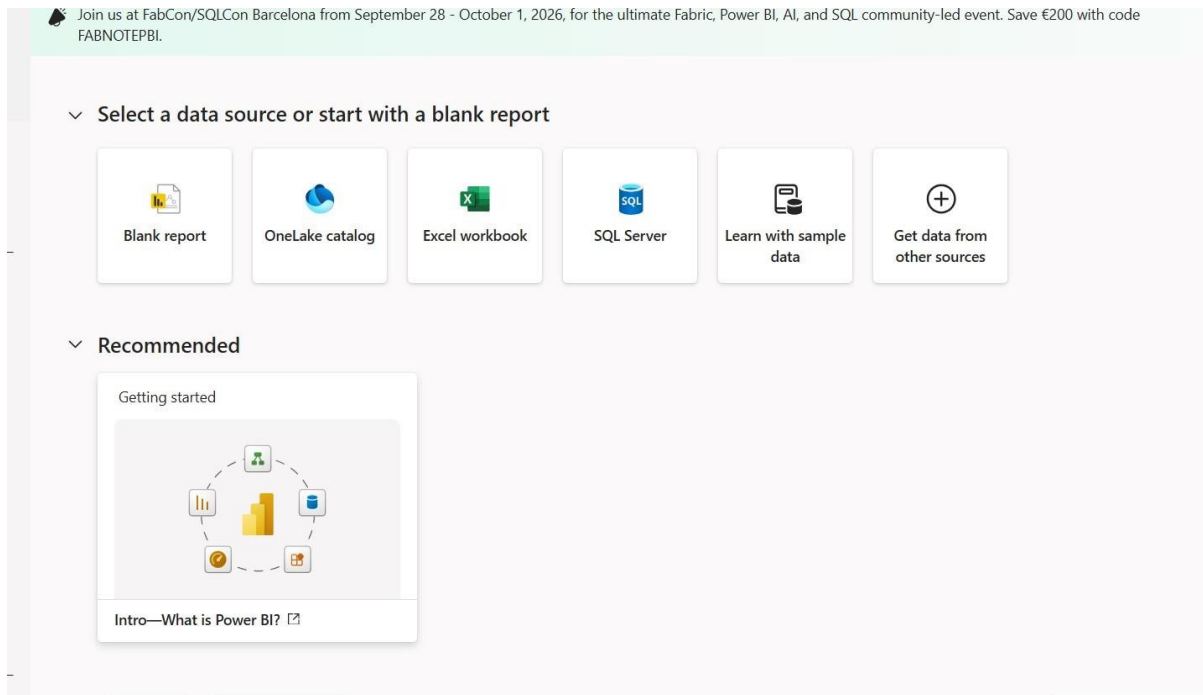


2) Import the csv file of the dataset

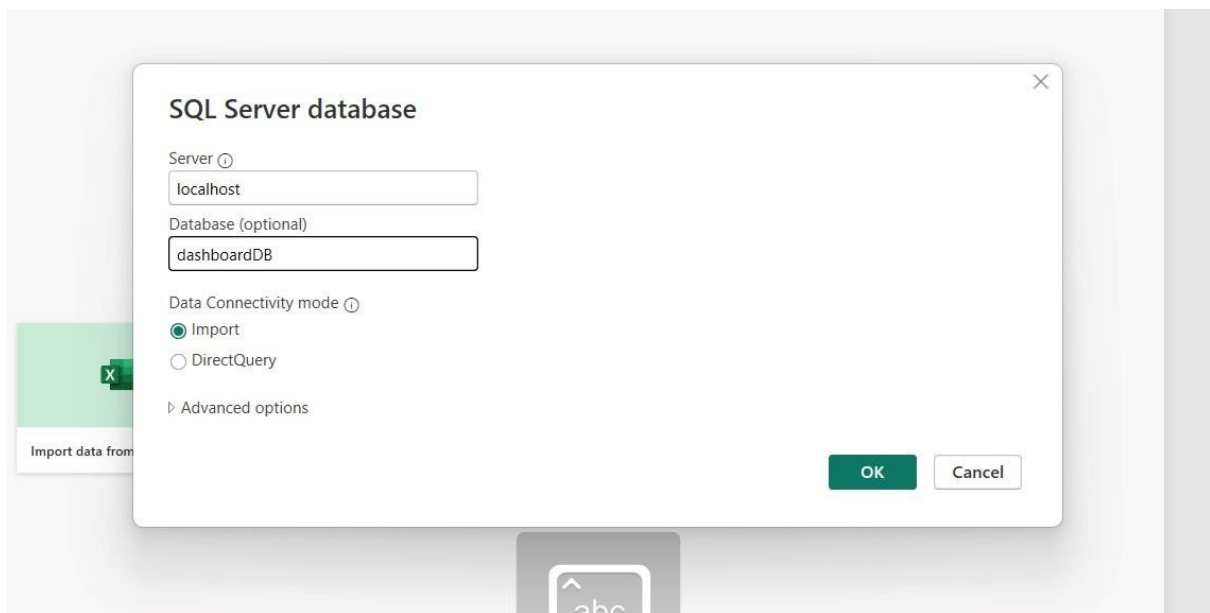


POWER BI

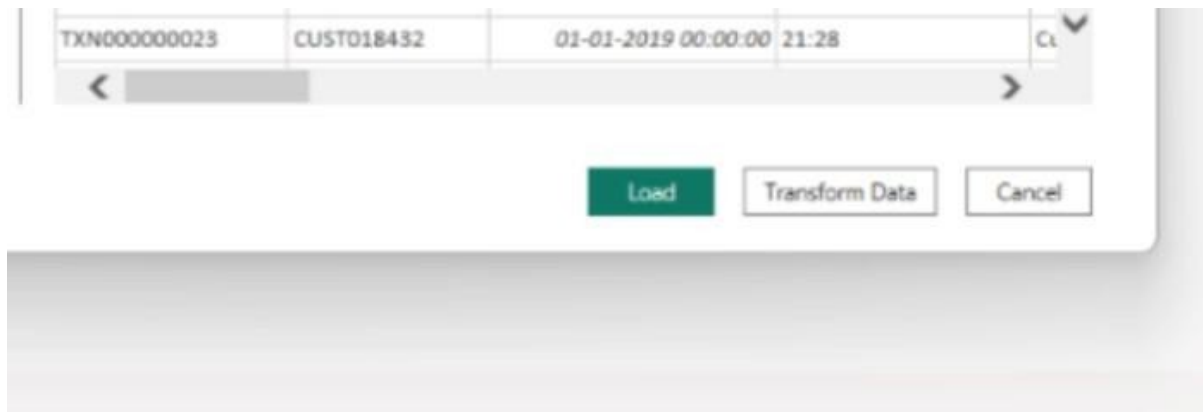
3) Now go to MICROSOFT POWER BI



4) Click SQL SERVER



5) Click OK and click transform data



6) Create DAX measures TOTAL LIKES :

```
1 Total Likes = SUM('Instagram_Analytics'[likes])
```

TOTAL POSTS:

```
1 Total Posts = COUNT('Instagram_Analytics'[post_id])
```

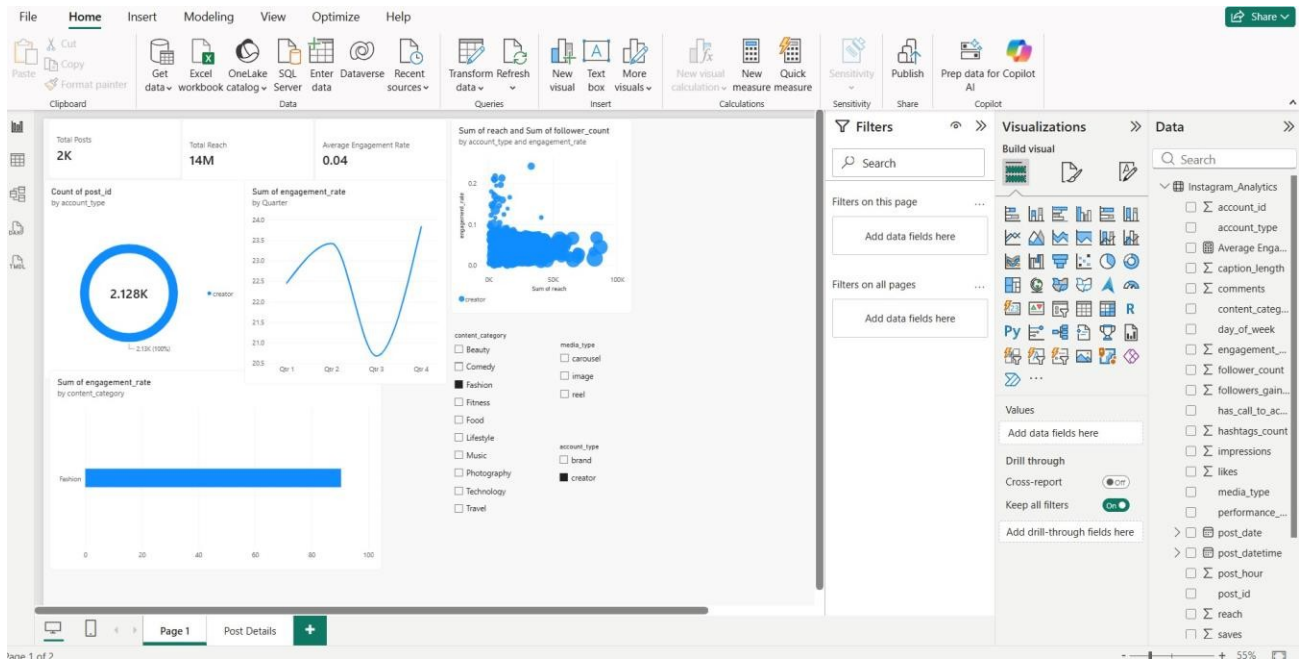
TOTAL REACH

```
1 Total Reach = SUM('Instagram_Analytics'[reach])
```

AVERAGE ENGAGEMENT RATE

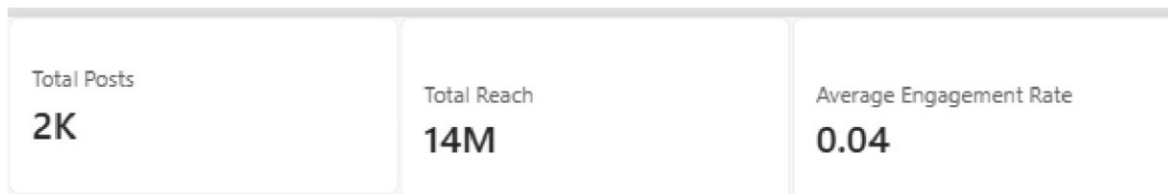
```
1 Average Engagement Rate = AVERAGE('Instagram_Analytics'[engagement_rate])
```

7) Power BI Final Executive Dashboard & Drill-Through

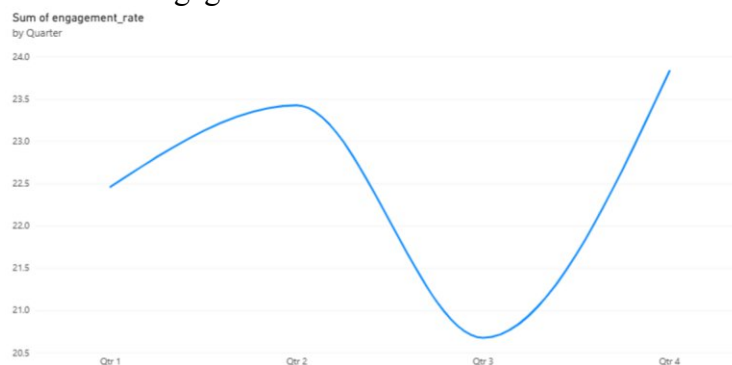


1. Executive Canvas:

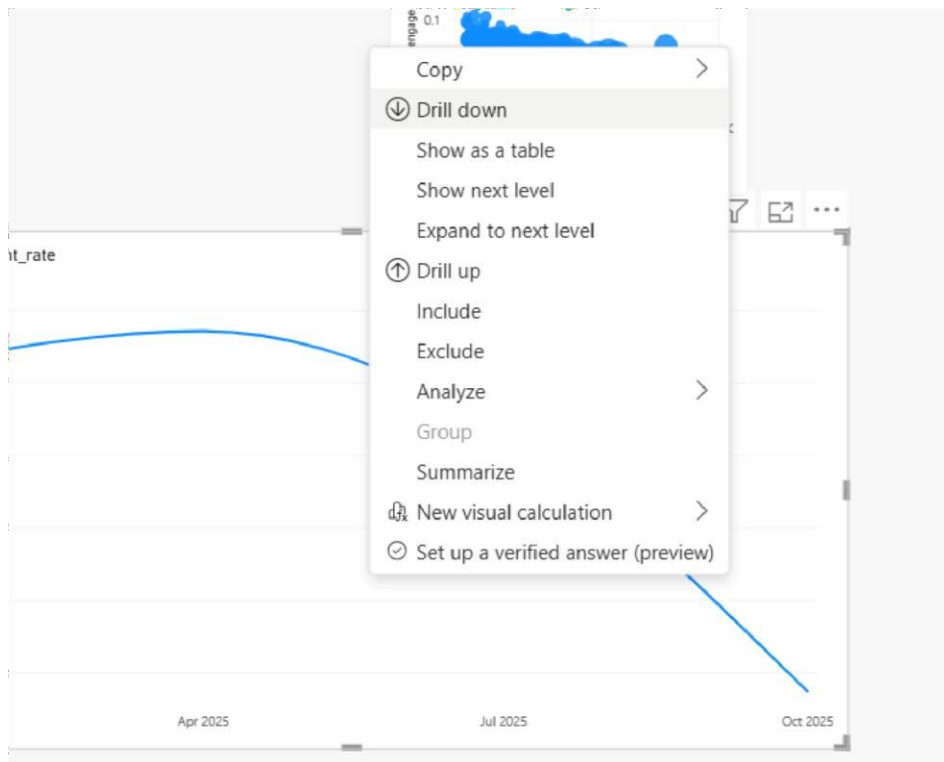
Place Card Visuals across the top: Total Likes, Total Transaction Reach, Total Shares, Average Engagement Rate



Add a Line Chart for sum of engagement rate .

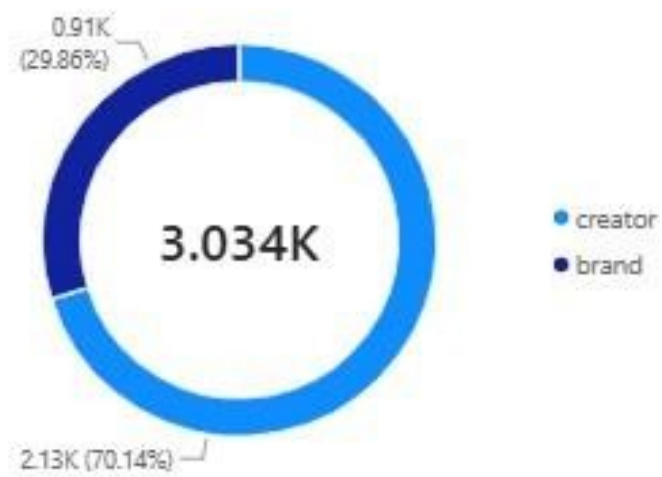


Enabled Drill Down



count by post id visualised by donut chart

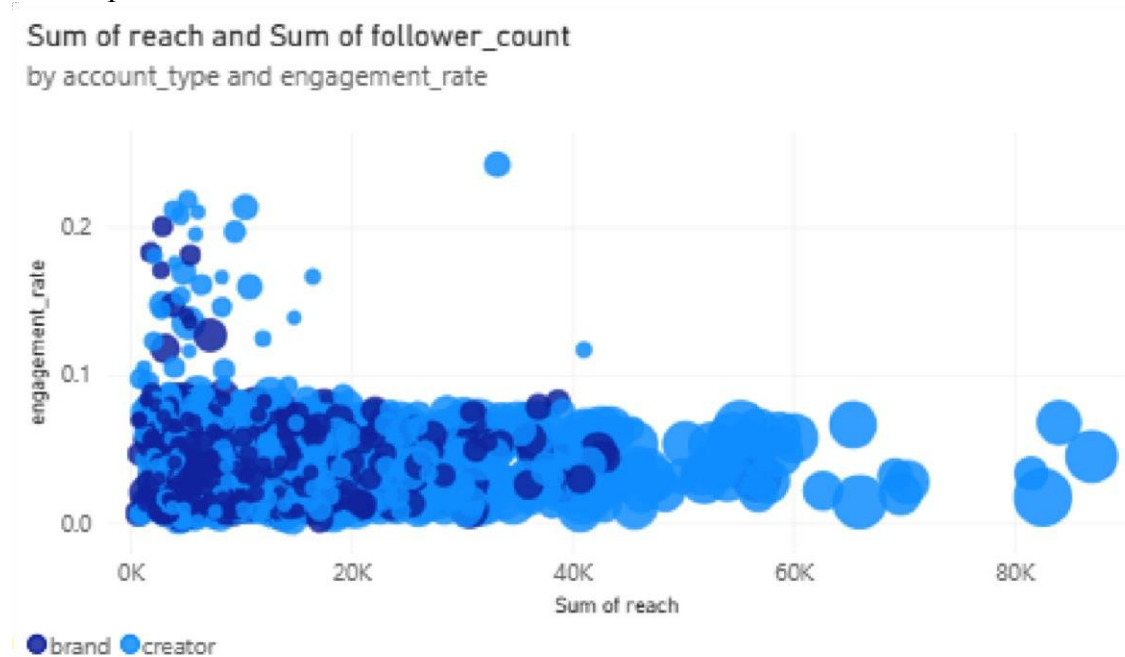
Count of post_id
by account_type



Create 3 slicers based on account type, content category and media type

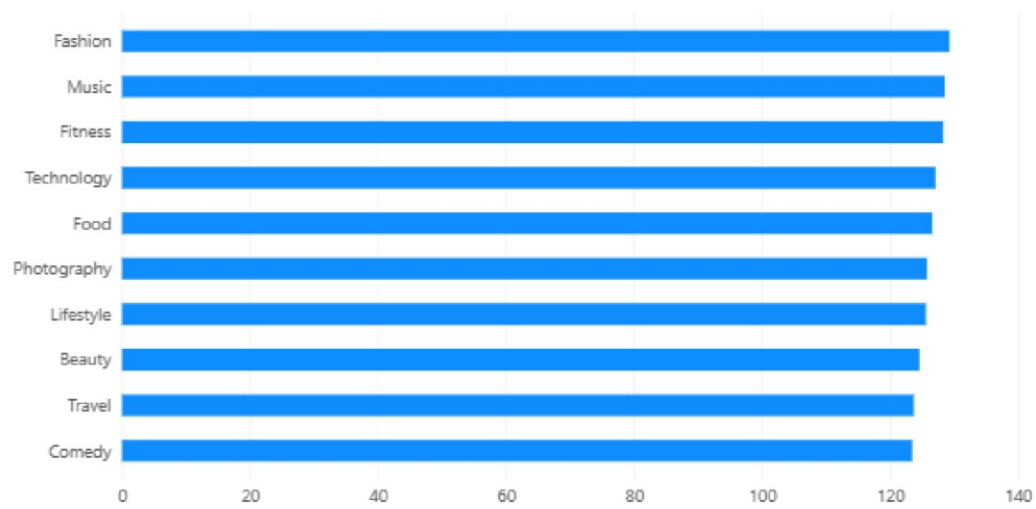


Scatter plot for sum of reach and sum of follower count



sum of engagement rate by content category

Sum of engagement_rate
by content_category



Enable drill-through

Values

Add data fields here

Drill through

Cross-report ☒

Keep all filters ☒

account_type is (All) ▼ × 🔒

post_id	account_type	media_type	content_category	Sum of likes	Sum of comments	Sum of shares	Sum of reach	Sum of impressions	Sum of engagement_rate
IG0000001	brand	reel	Technology	194	5	7	4327	6230	0.04
IG0000002	creator	image	Fitness	449	10	21	7451	8268	0.07
IG0000003	brand	reel	Beauty	114	2	1	1639	2616	0.05
IG0000004	creator	carousel	Music	91	0	7	2877	3171	0.03
IG0000005	creator	reel	Technology	154	8	5	5350	8503	0.02
IG0000006	brand	carousel	Photography	149	3	4	2668	3472	0.05
IG0000007	creator	carousel	Food	371	9	14	5018	7871	0.06
IG0000008	brand	reel	Technology	18	1	0	1078	1357	0.02
IG0000009	brand	reel	Beauty	132	4	7	1900	2436	0.07
IG0000010	creator	reel	Music	101	2	3	2155	2942	0.04
IG0000011	creator	reel	Fitness	418	11	21	14307	20978	0.02
IG0000012	creator	carousel	Beauty	49	6	2	762	1098	0.05
IG0000013	brand	image	Beauty	111	1	7	4364	6329	0.02
IG0000014	brand	image	Food	183	6	10	3997	6107	0.04
IG0000015	brand	reel	Lifestyle	333	10	15	5240	6377	0.06
IG0000016	creator	reel	Music	271	8	13	5981	8127	0.04
IG0000017	creator	image	Travel	138	1	4	4489	5263	0.03
IG0000018	creator	carousel	Beauty	261	5	16	6267	9464	0.03
IG0000019	creator	image	Lifestyle	120	9	5	8187	11264	0.01
IG0000020	brand	carousel	Travel	39	0	2	3345	4755	0.01
IG0000021	brand	image	Music	303	0	12	3950	5435	0.07
IG0000000	brand	reel	Food	444	17	24	20462	33264	0.04
Total				8629320	255649	432784	188167991	254000108	1,263.18

TABLEAU

Import the data from sql server

Tableau - Book1

File Data Server Help

Upgrade to a paid license to share and publish your work

Start a Cloud Trial Buy Tableau

Microsoft SQL Server

General Initial SQL Advanced

Server
localhost

Database
dashboardDB

Authentication
Use Windows Authentication (preferred)

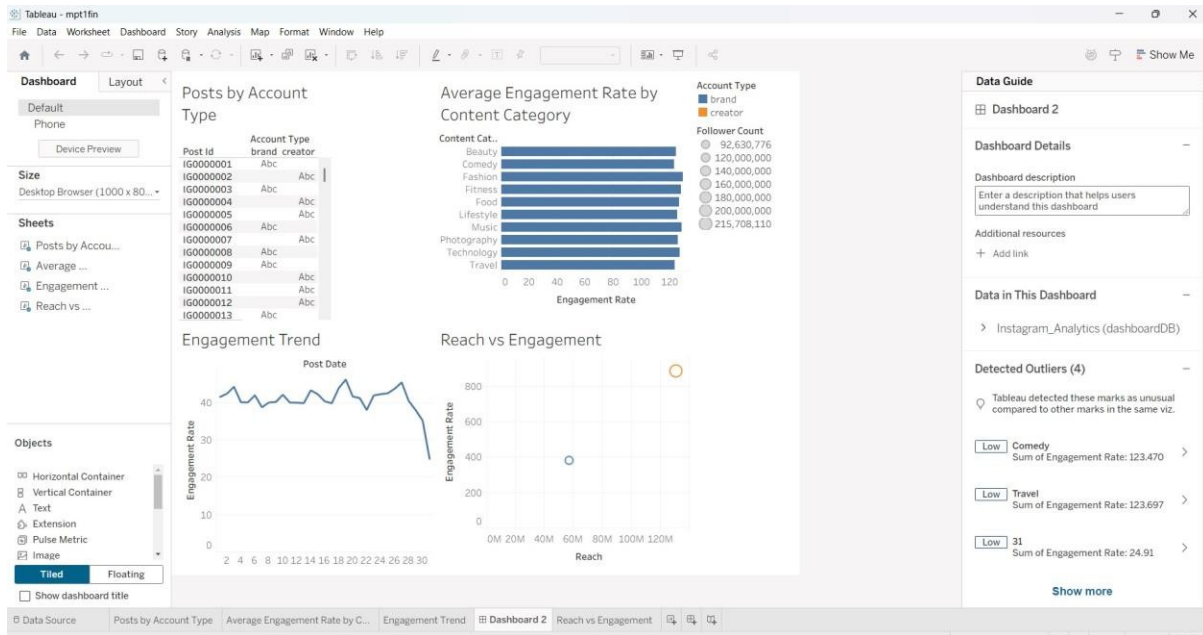
☐ Require SSL

☐ Read uncommitted data

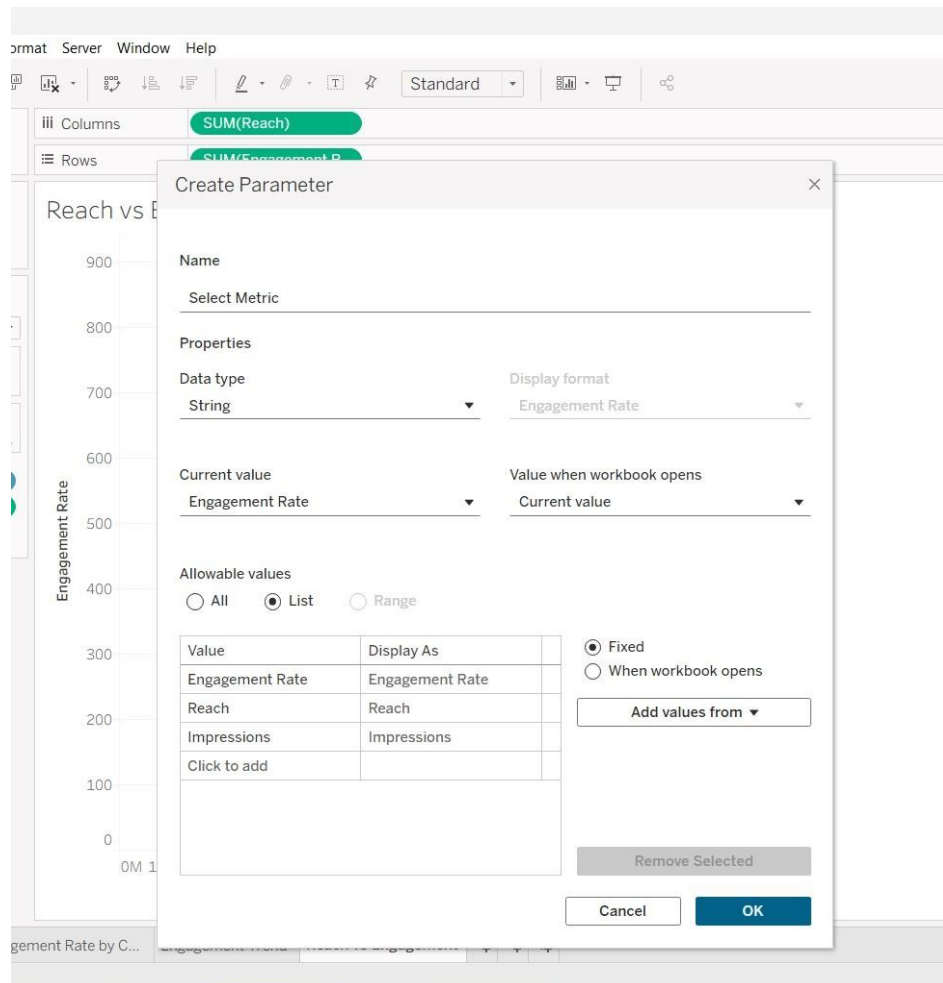
Sign In

Quick Start

FINAL DASHBOARD :



Build parameters



Selected Metric

×

CASE [Select Metric]
WHEN "Engagement Rate" THEN [engagement_rate]
WHEN "Reach" THEN [reach]
WHEN "Impressions" THEN [impressions]
END

The calculation is valid.

Apply

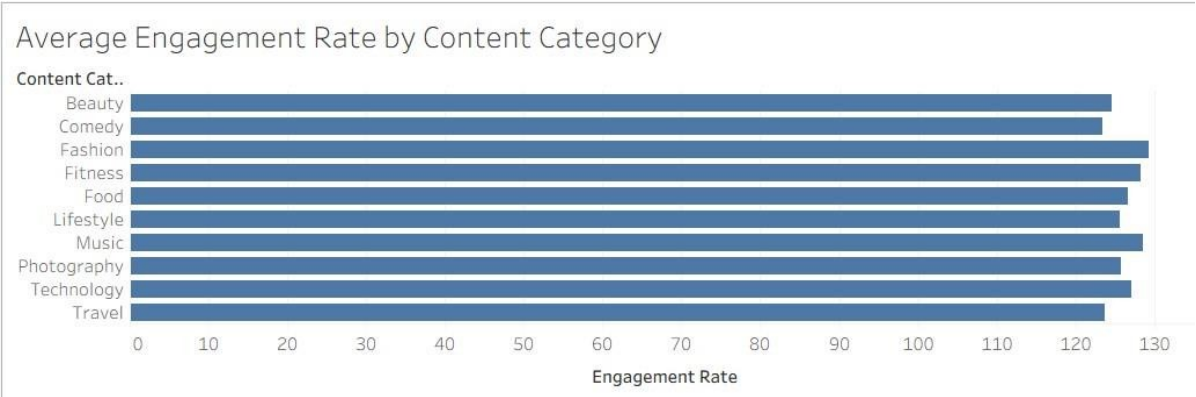
OK

Create a table to display post by account type

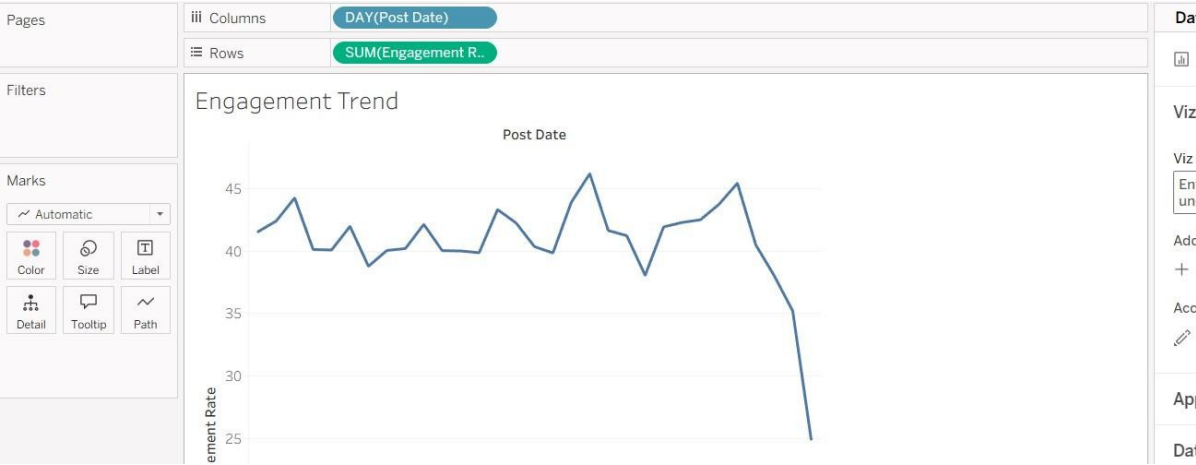
Posts by Account Type

Post Id	Account Type	brand creator
IG0000001	Abc	
IG0000002		Abc
IG0000003	Abc	
IG0000004		Abc
IG0000005		Abc
IG0000006	Abc	
IG0000007		Abc
IG0000008	Abc	
IG0000009	Abc	
IG0000010		Abc
IG0000011		Abc
IG0000012		Abc
IG0000013	Abc	

To display average engagement rate by content category

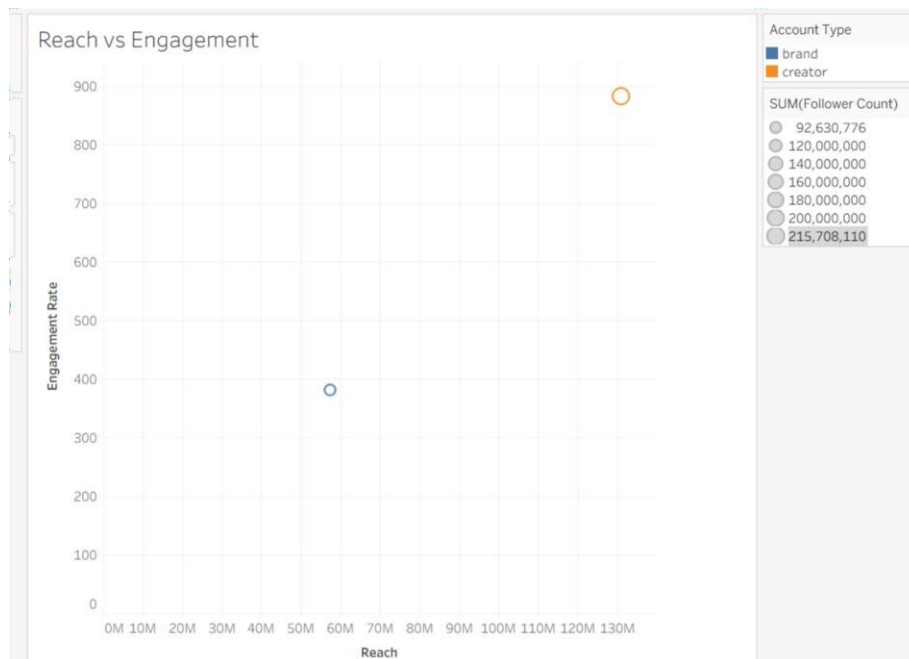


Line chart to visualise engagement trend with drill down



Year	2015
Quarter	Q2
Month	May
Day	8
More	▶

Reach VS Engagement Rate



Observed Insights

1. The dashboard contains approximately 30K Instagram posts with a total reach of 188M.
The average engagement rate is approximately 0.04.
This indicates a large volume of content and significant audience reach.
2. Creator accounts contribute a larger share of posts than brand accounts.
Creators account for approximately 69.82% of the posts, while brands account for about 30.18%.
This shows that creator accounts dominate the dataset.
3. Engagement varies across content categories, with Fashion showing the highest engagement.
Music and Fitness also show relatively high engagement compared with other categories. Comedy and Travel appear comparatively lower in the displayed chart.
4. Engagement rate changes across the quarters, reaching its highest level in Q4.
The trend increases from Q1 to Q2, decreases during Q3, and rises again in Q4.
This indicates noticeable variation in engagement performance throughout the year.

Ex.no:9	MINI PROJECT – HEALTHCARE ANALYTICS

Domain and Problem Statement

Domain: Healthcare Data Analytics & Hospital Performance Analysis

Problem Statement:

Healthcare organizations generate large volumes of hospital-related data but often lack clear insights into hospital distribution, ratings, reviews, and regional patterns. This project addresses these challenges by transforming hospital data into structured business intelligence using Power BI and Tableau, enabling analysis of hospital distribution across states and cities, average ratings, review patterns, and overall hospital performance to support data-driven healthcare analysis.

Dataset Description

Dataset Name: Hospital Analytics

Dataset Granularity: Each row represents a single hospital.

Key Entities & Attributes:

- **Hospital & Location:** id, City, State, District
- **Geographical Information:** Density, Latitude, Longitude
- **Hospital Performance:** Rating, Number of Reviews

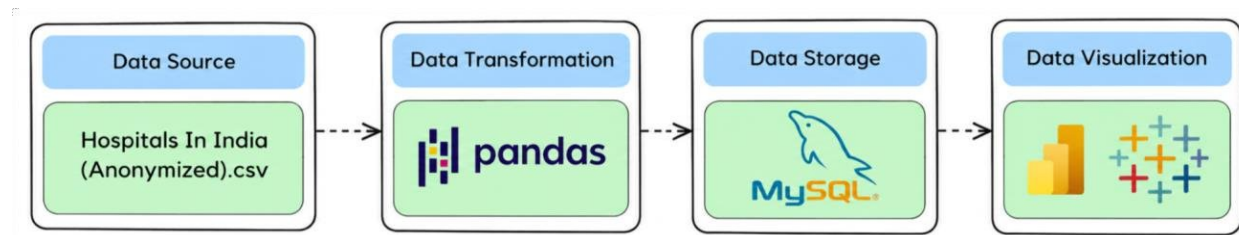
Data Source:

Primary Source: Hospital Analytics Dataset prepared for healthcare and hospital performance analysis.

Storage Infrastructure:

Cleaned and pre-processed hospital records maintained in a CSV dataset and imported into Microsoft SQL Server, Power BI and Tableau for interactive dashboard development and analysis.

Architecture:



Program:

1. Python – Data Cleaning &

Transformation import pandas as pd

```
df = pd.read_csv("Hospitals In India  
(Anonymized).csv") hospitals =  
df.copy()
```

```
hospitals["City"] =  
hospitals["City"].fillna("Unknown") hospitals["State"]  
= hospitals["State"].fillna("Unknown")  
hospitals["District"] =  
hospitals["District"].fillna("Unknown")  
numeric_columns = [
```

```
    "Density",  
    "Latitude",  
    "Longitude",  
    "Rating",  
    "Number of Reviews"
```

```
]
for column in numeric_columns:  
    hospitals[column] =
```

```
hospitals[column].fillna(0)
```

```
hospitals.drop_duplicates(inplace  
=True) hospitals.columns =
```

```
( hospitals.columns
```

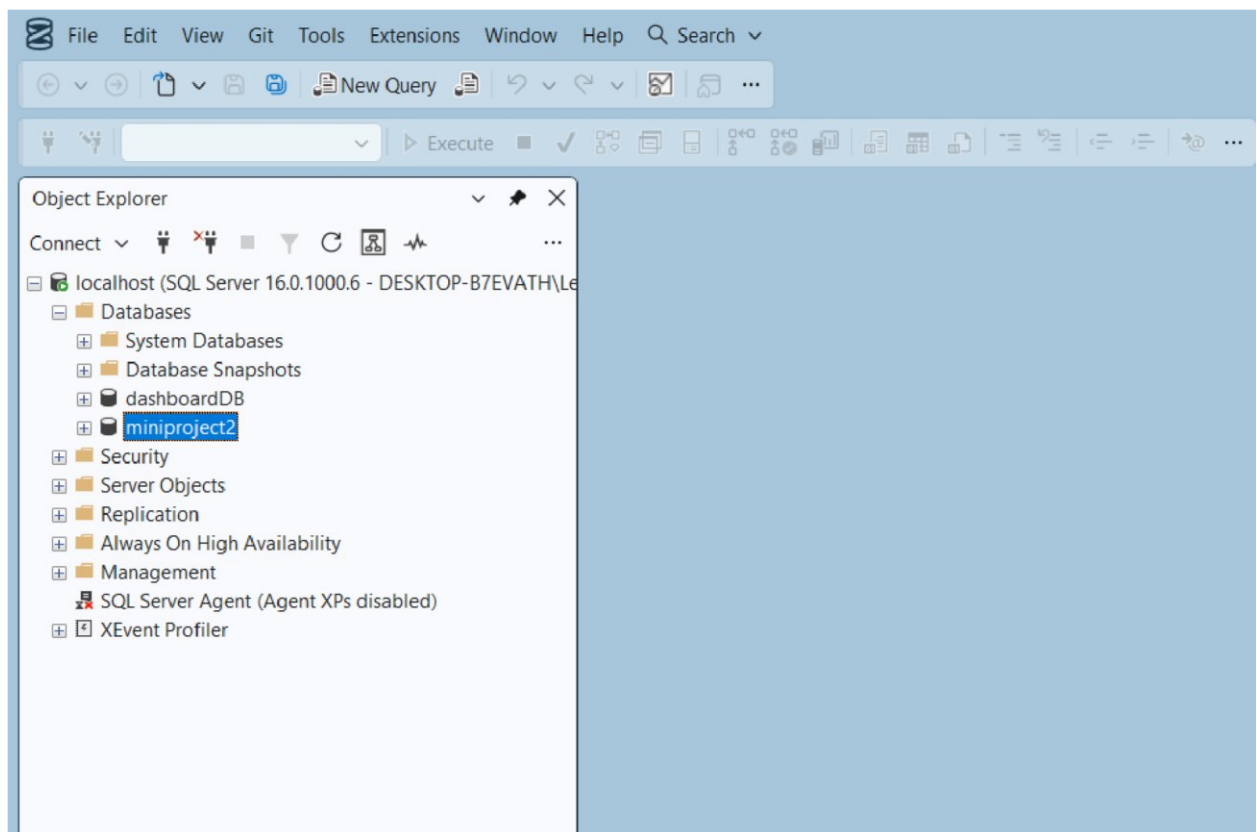
```
    .str.lower()  
    .str.strip()
```

```
.str.replace(" ", "_"))  
print("Original Shape:",  
df.shape) print("Cleaned  
Shape:", hospitals.shape)  
print("\nDataset  
Preview:")  
print(hospitals.head())  
print("\nDataset  
Information:")  
print(hospitals.info())
```

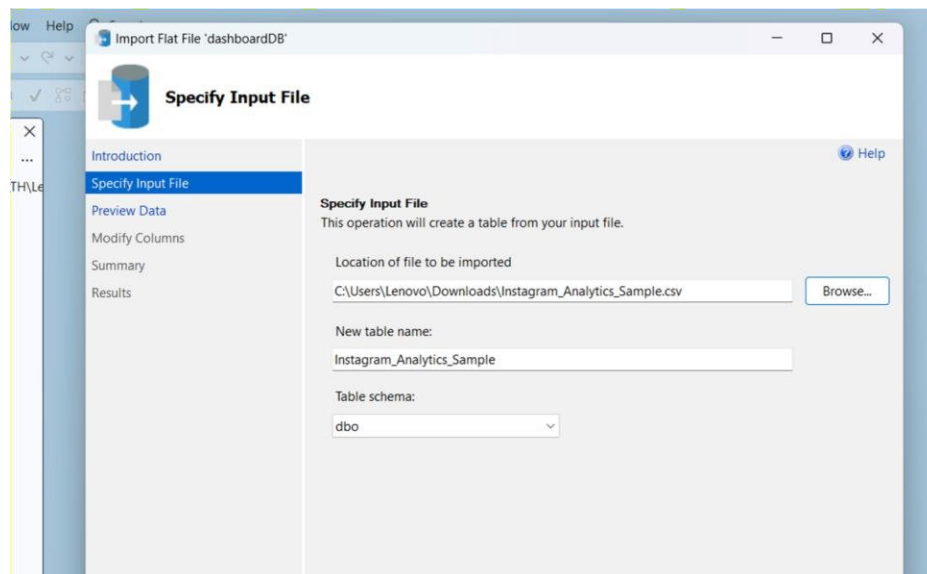
2. SQL SERVER CONNECTIVITY

INSTALL SQL SERVER MANAGEMENT STUDIO

1) Create database named “miniproject2”

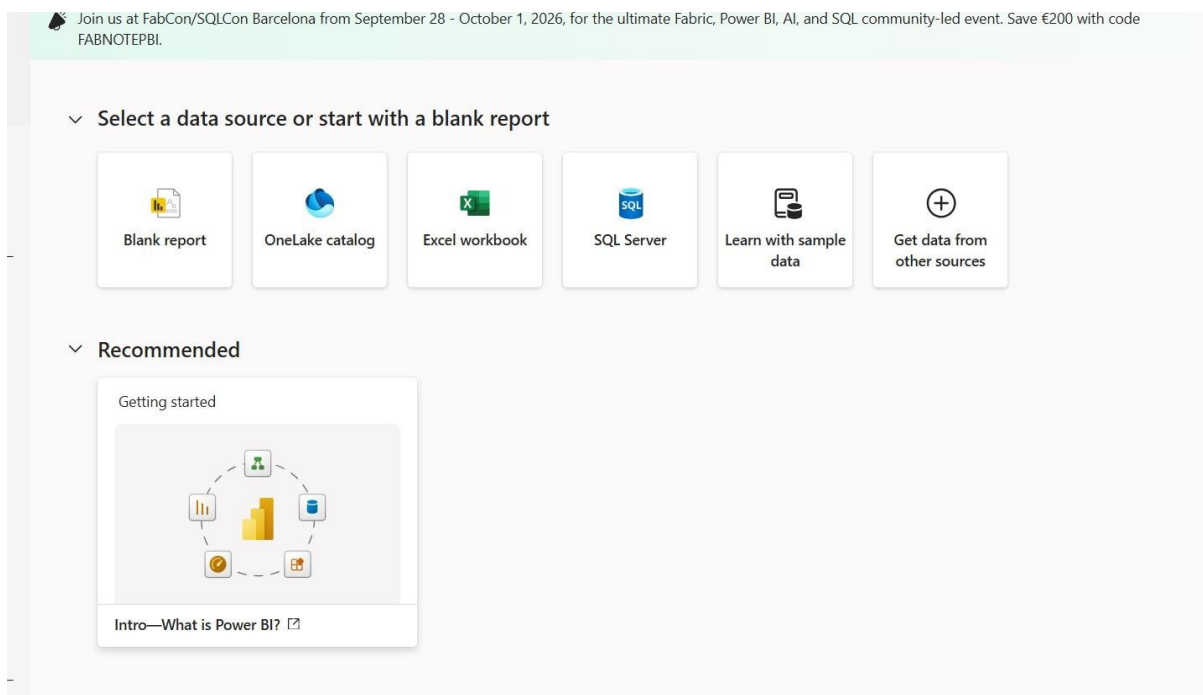


2) Import the csv file of the dataset

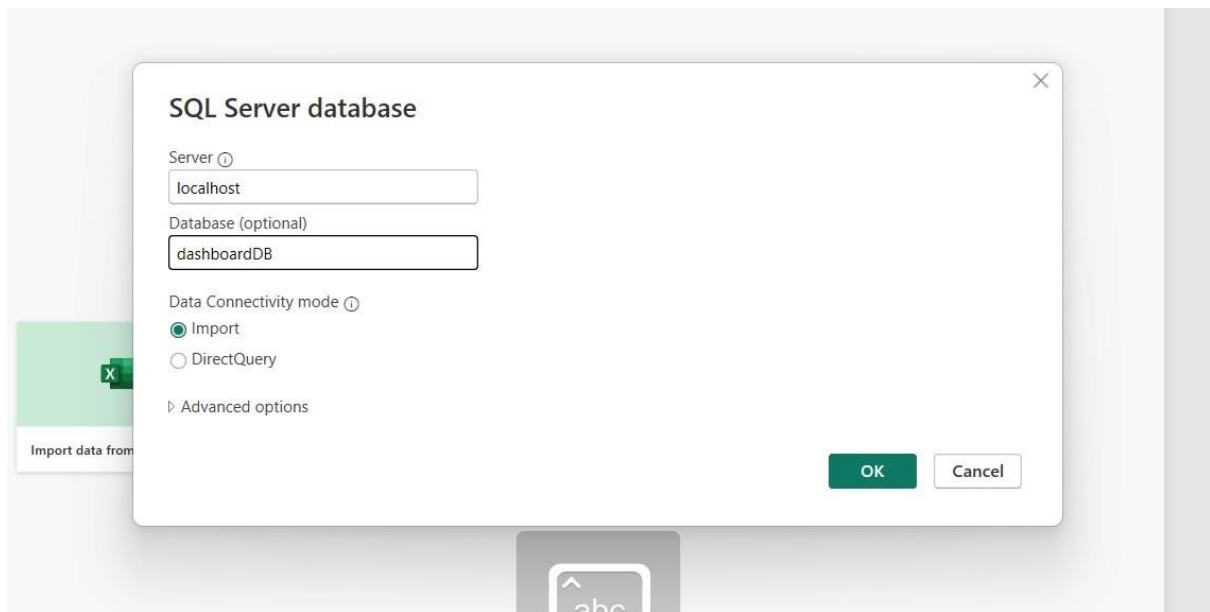


POWER BI

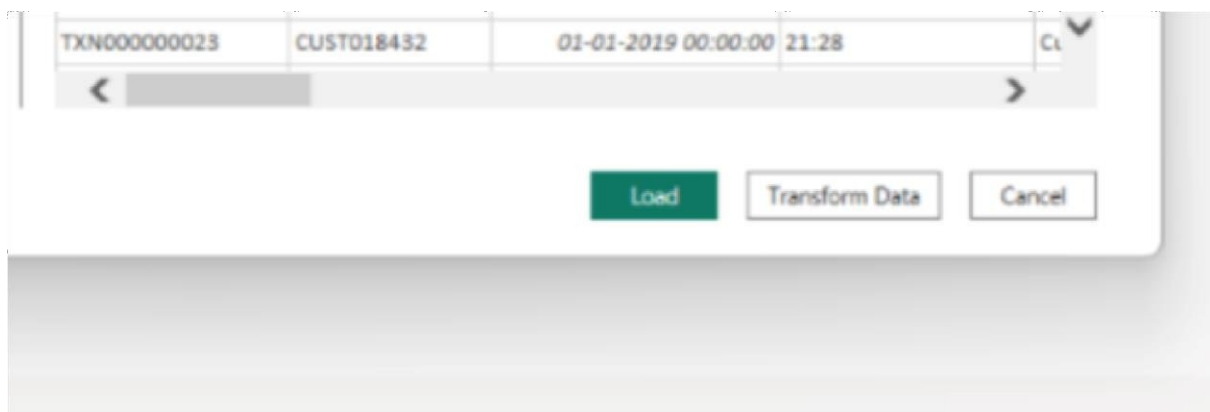
3) Now go to MICROSOFT POWER BI



4) Click SQL SERVER



5) Click OK and click transform data



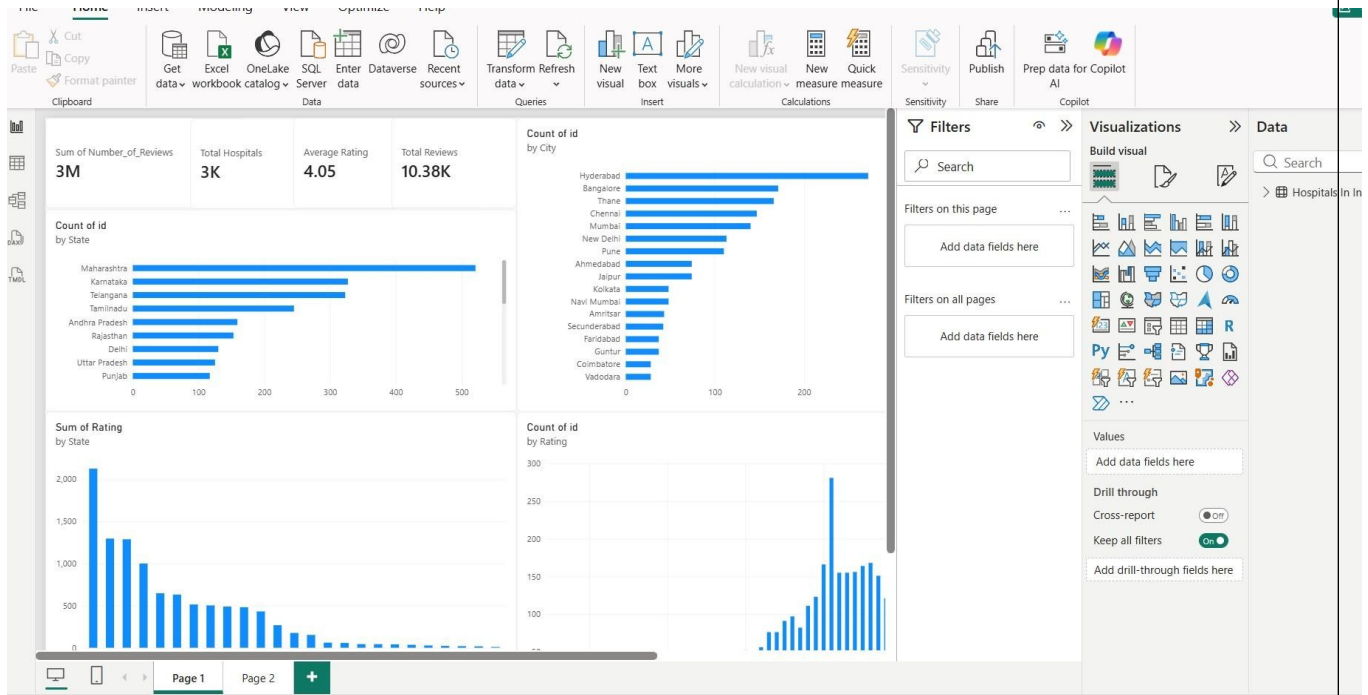
6) Create DAX measures TOTAL HOSPITALS :

```
1 Total Hospitals = COUNT('Hospitals In India (Anonymized)'[id])
```

AVERAGE RATING :

```
1 Average Rating = AVERAGE('Hospitals In India (Anonymized)'[rating])
```

7) Power BI Final Executive Dashboard & Drill-Through



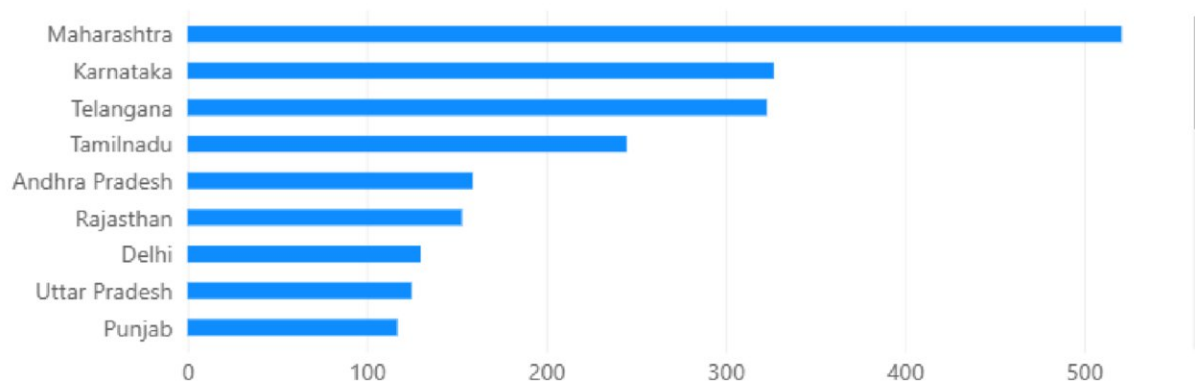
1. Executive Canvas:

Place Card Visuals across the top:



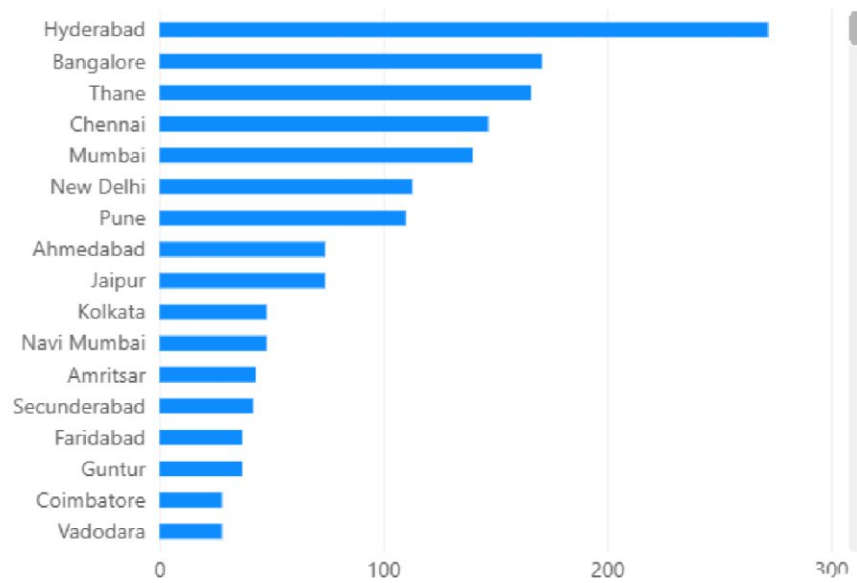
Visualise number of hospitals in each state

Count of id
by State



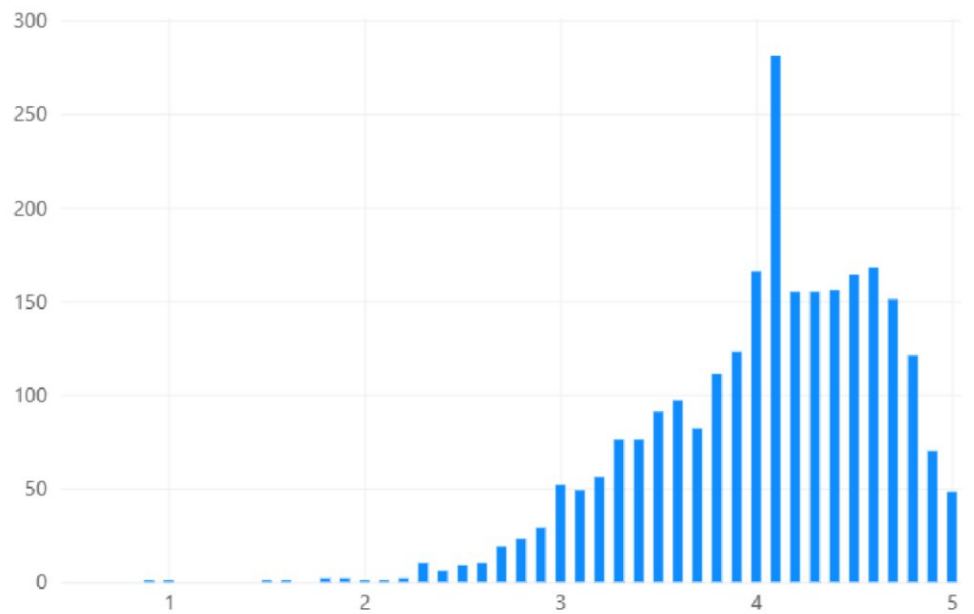
count number of hospitals by city

Count of id
by City



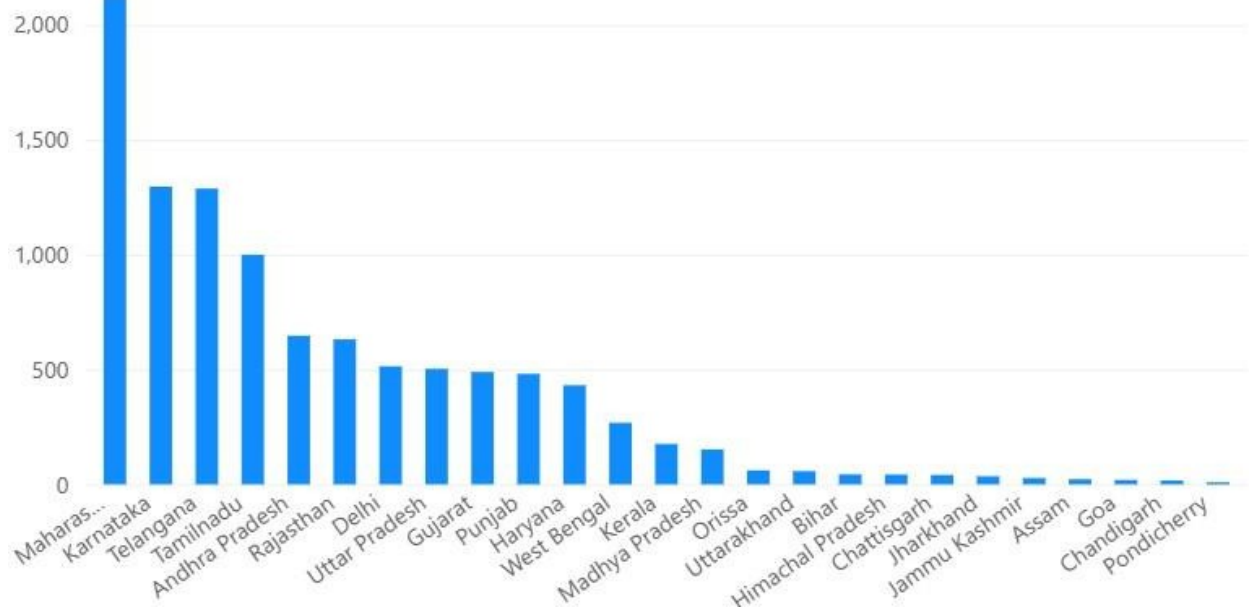
Count of id by rating

Count of id
by Rating



Total rating by states

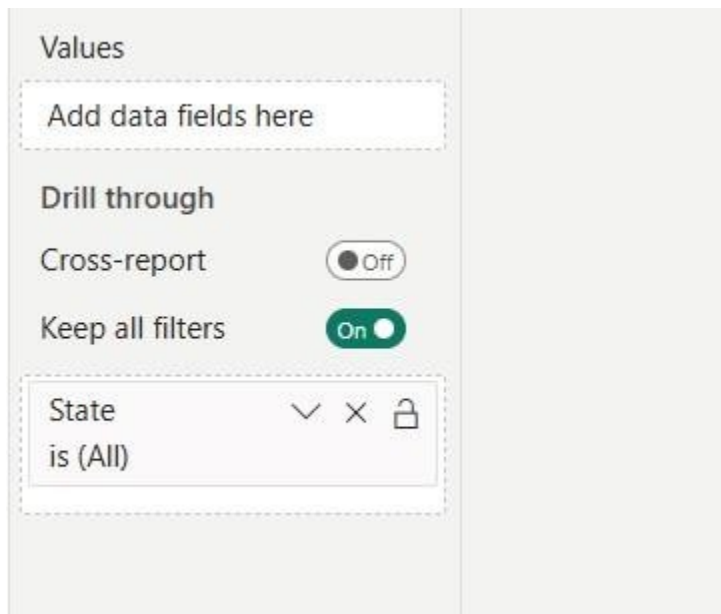
Sum of Rating
by State



Create 2 slicers by state and city

State	City
☐ Andhra Pradesh	☐ Agra
☐ Assam	☐ Ahmedabad
☐ Bihar	☐ Ahmednagar
☐ Chandigarh	☐ Ajmer
☐ Chattisgarh	☐ Aligarh
☐ Delhi	☐ Allahabad
☐ Goa	☐ Aluva
☐ Gujarat	☐ Alwar
☐ Haryana	☐ Ambala
☐ Himachal Pradesh	☐ Amritsar
☐ Jammu Kashmir	☐ Anantpur
☐ Jharkhand	☐ Aurangabad
☐ Karnataka	☐ Bahadurgarh
☐ Kerala	☐ Balotra
☐ Madhya Pradesh	☐ Bangalore
☐ Maharashtra	☐ Banswara
☐ Orissa	☐ Bantwal
☐ Pondicherry	☐ Baramati
☐ Punjab	☐ Bareilly
☐ Rajasthan	☐ Barmer
☐ Tamil Nadu	☐ Behrampur
☐ Telangana	☐ Belgaum
☐ Uttar Pradesh	☐ Belguam
☐ Uttarakhand	☐ Bellary
☐ West Bengal	☐ Berhampur

Implement drill through



City Total Hospitals Total Reviews

Agra	3	12.10
Ahmedabad	74	315.90
Ahmednagar	2	6.90
Ajmer	3	12.50
Aligarh	1	4.50
Allahabad	3	12.30
Aluva	1	3.50
Alwar	7	29.70
Ambala	13	53.30
Amritsar	43	172.30
Anantpur	2	9.00
Aurangabad	6	24.00
Bahadurgarh	2	6.90
Balotra	1	4.90
Bangalore	171	687.80
Banswara	1	4.00
Banhal	1	3.20
Baramati	1	4.00
Bareilly	14	52.40
Barmer	1	4.10
Belgaum	1	4.00
Total	2566	10,379.80

Filters

Search

Filters on this page

Add data fields here

Filters on all pages

Add data fields here

Visualizations

Build visual

Search

Hospital

Values

Add data fields here

Drill through

Cross-report ☐ Off

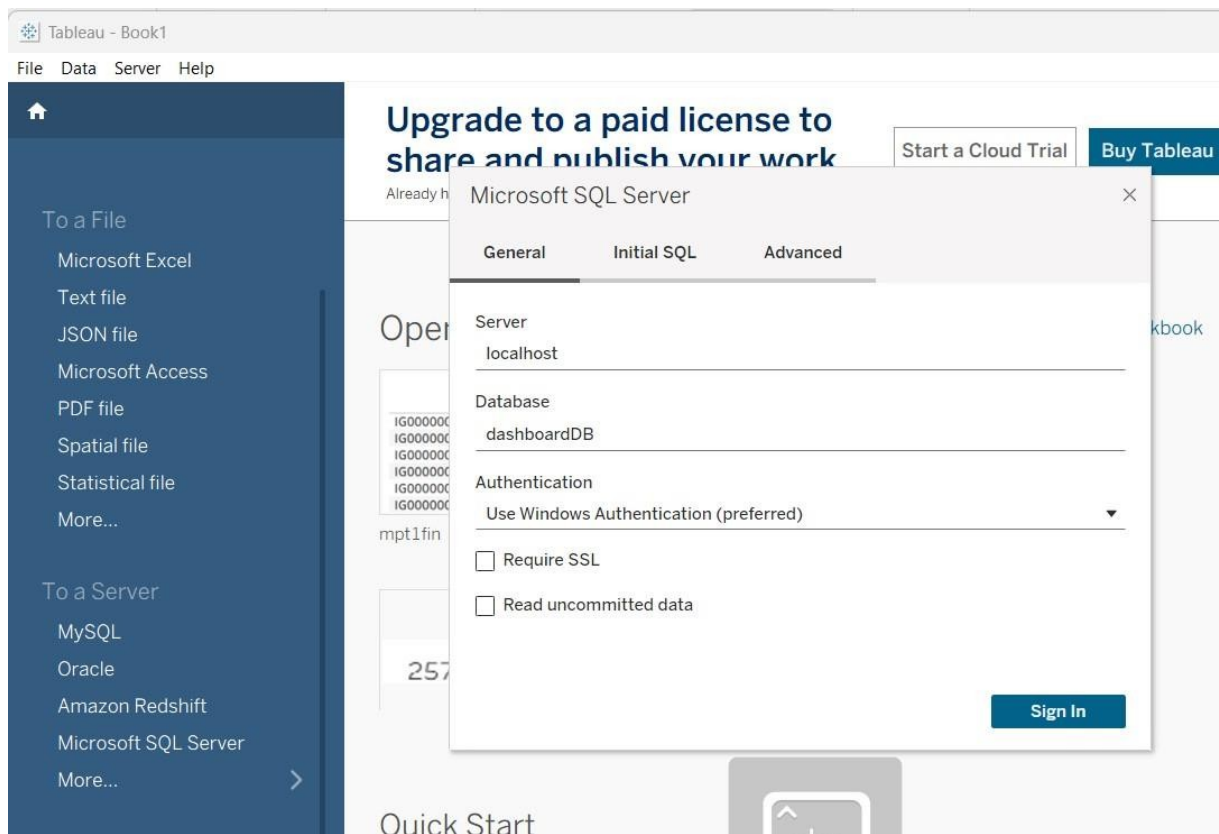
Keep all filters ☒ On

State is (All) ▼ × 🔒

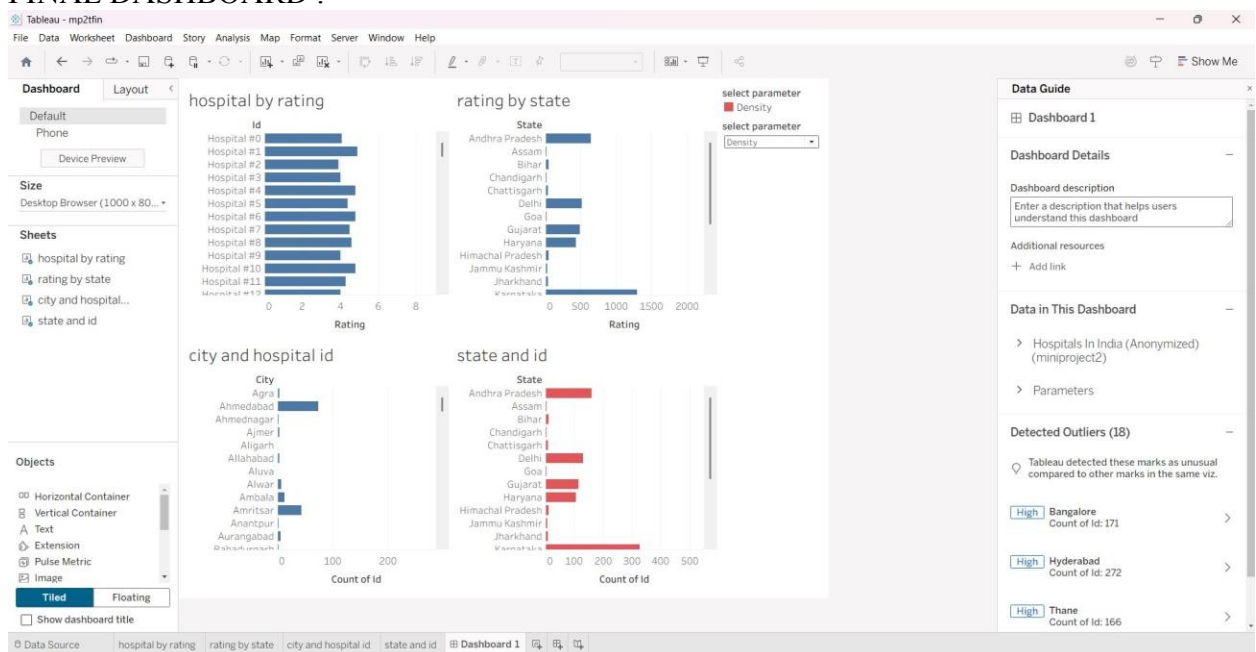
Page 1 Page 2 +

TABLEAU

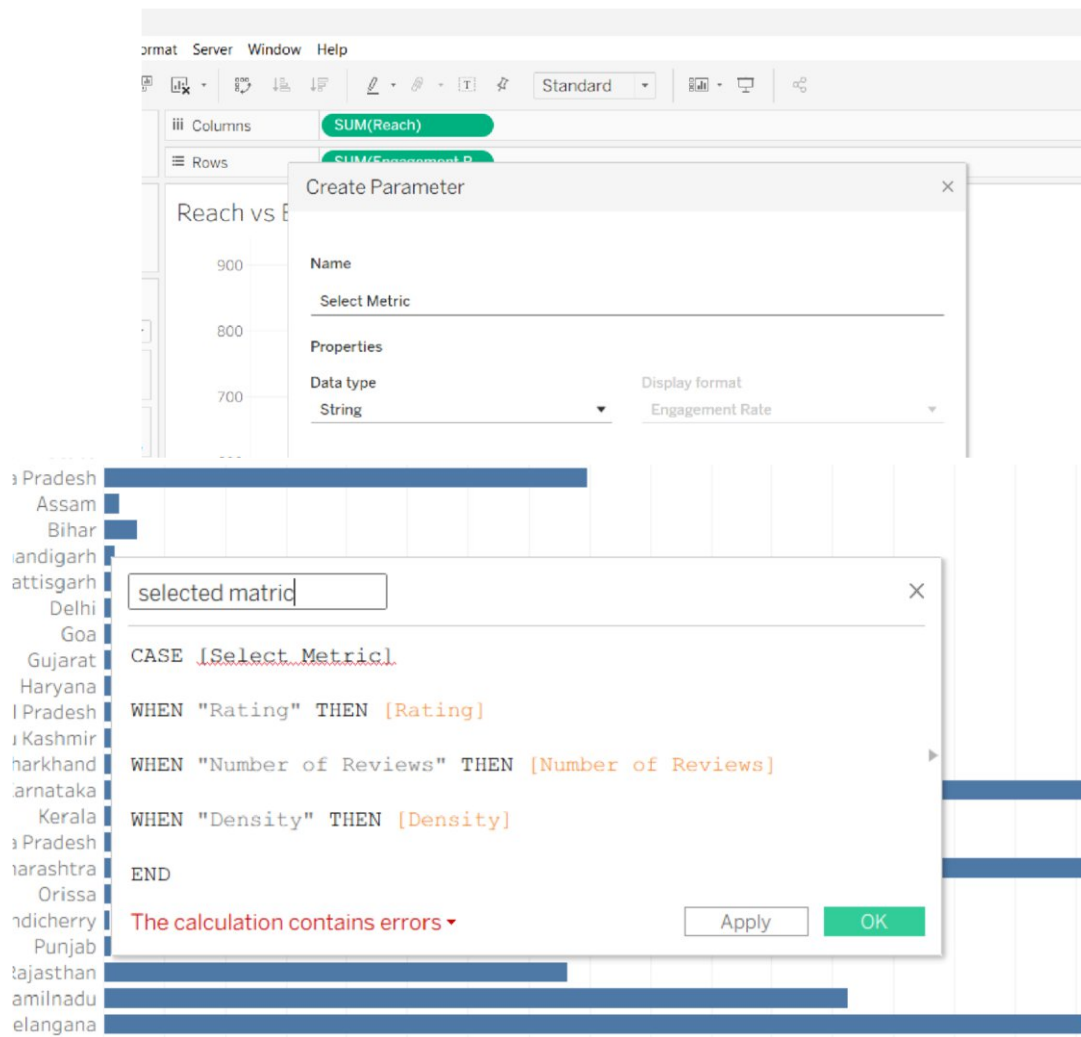
Import the data from sql server



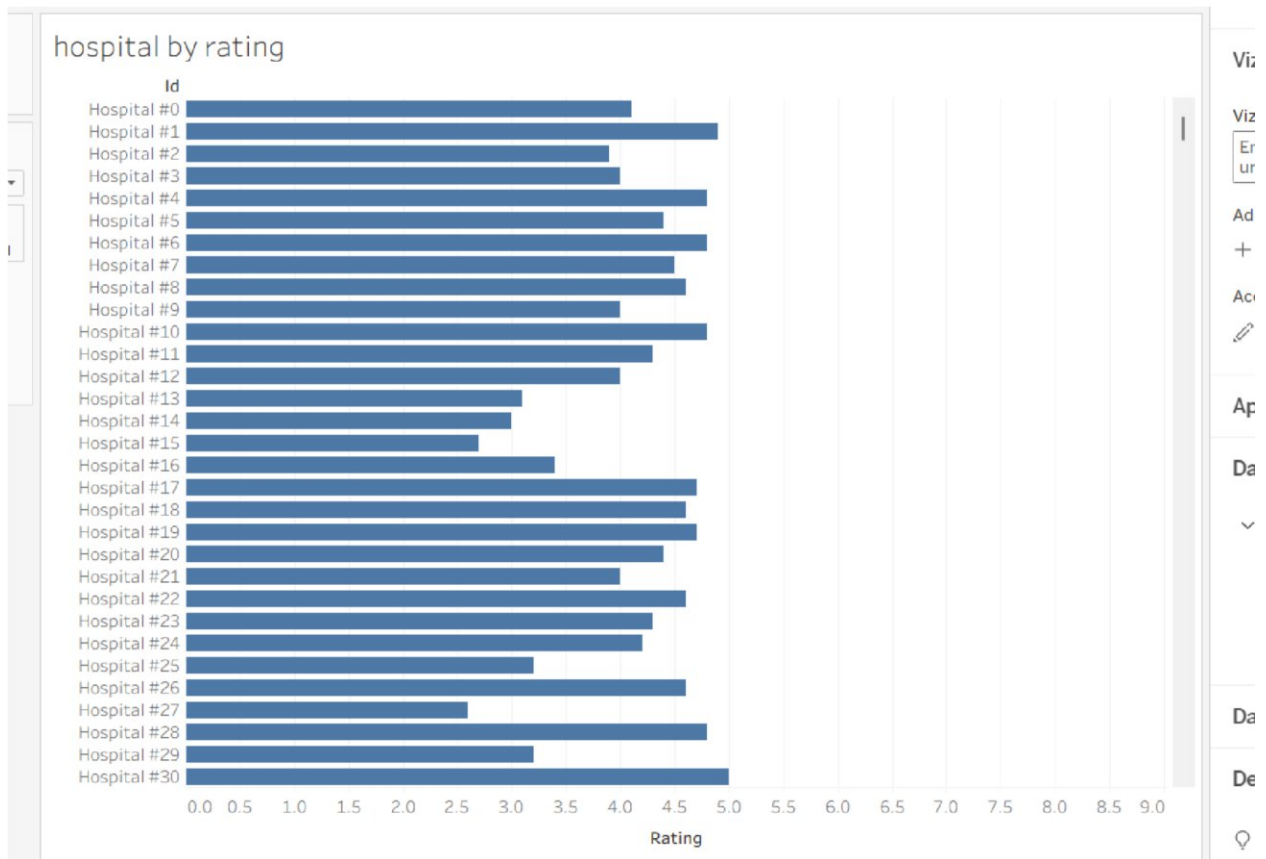
FINAL DASHBOARD :



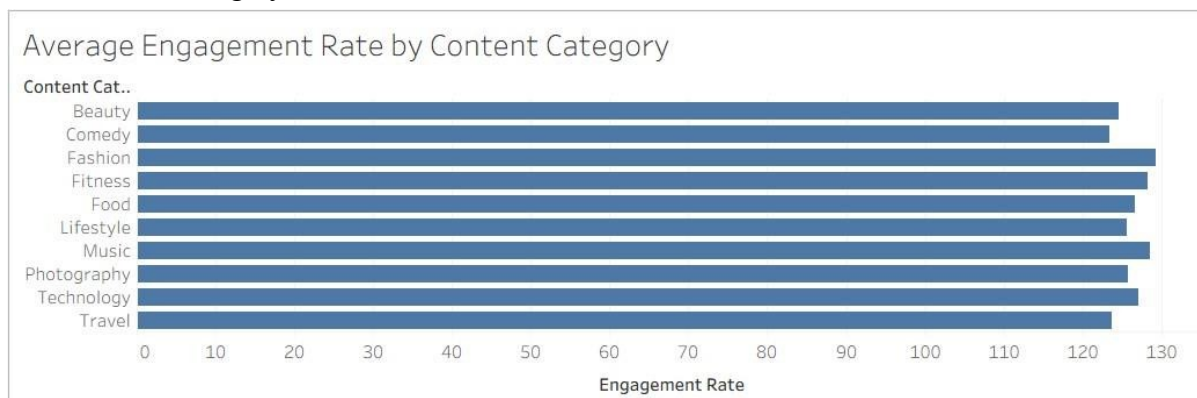
Build parameters



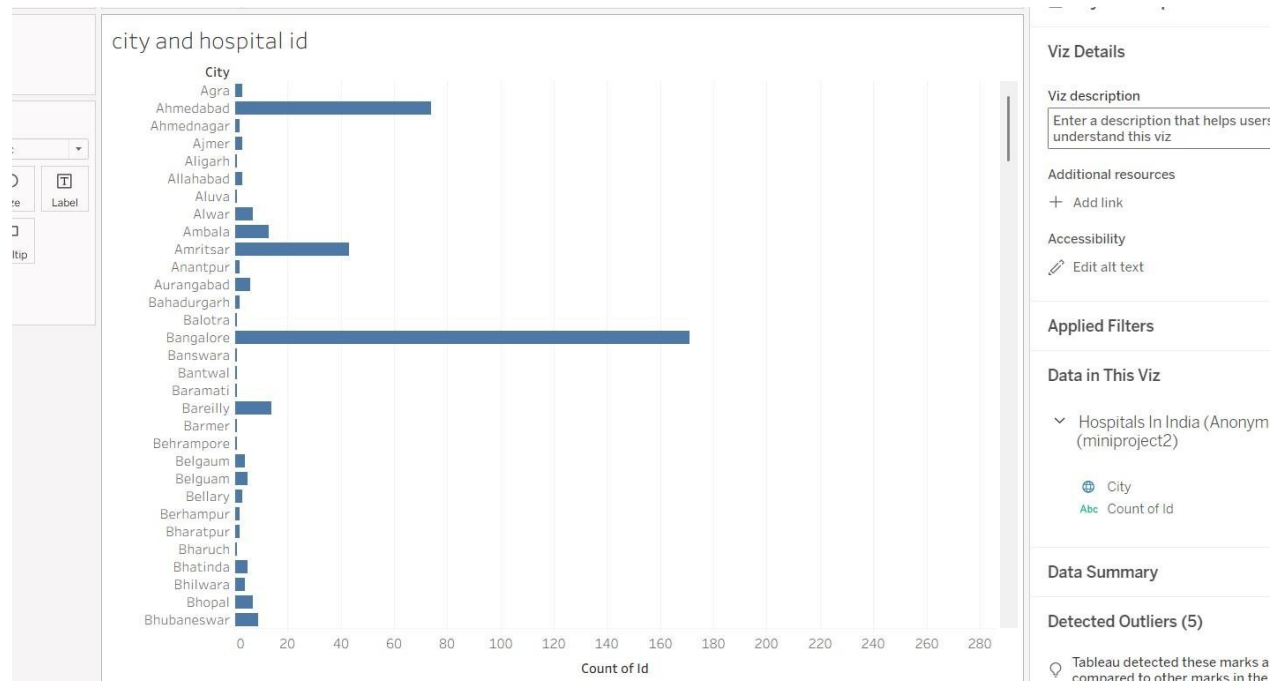
To visualise hospital by rating



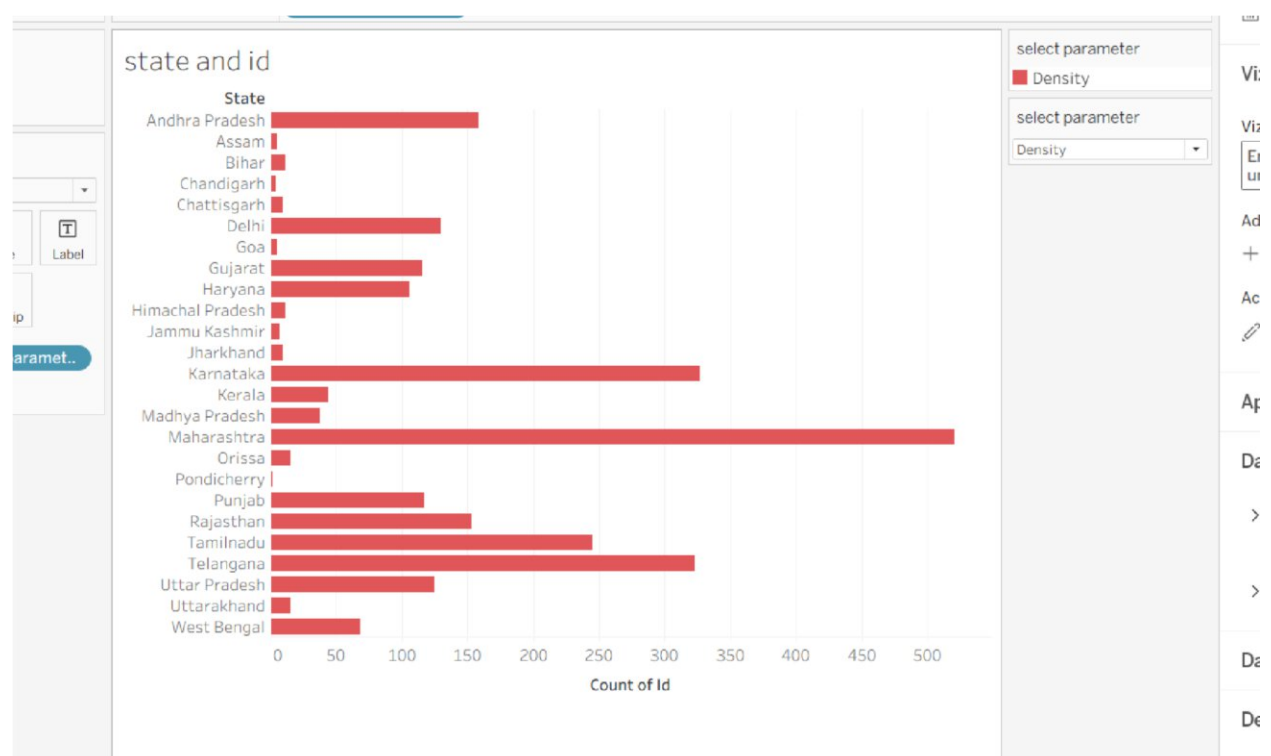
To visualize rating by state



To visualize number of hospitals in each city



To visualise number of hospitals in each state



Observed Insights

1. Hospital Ratings

Most hospitals have ratings between 3 and 5, indicating generally moderate to good ratings. Hospital #1 has the highest rating among the displayed hospitals, close to 5. The ratings are relatively consistent across the hospitals shown in the dashboard.

2. Rating by State

Karnataka has the highest rating-related value among the states displayed, followed by Andhra Pradesh. Delhi, Gujarat, and Haryana also show comparatively higher values than most other states. Several states such as Bihar, Assam, and Chandigarh have very low values in comparison.

3. Hospital Distribution by City

Ahmedabad has the highest hospital count among the cities displayed in the dashboard. Amritsar also has a noticeable number of hospitals compared with most other listed cities. Several cities such as Agra, Ajmer, Aligarh, and Allahabad have relatively fewer hospitals.

4. Hospital Distribution by State

Karnataka has the highest number of hospital IDs, followed by Andhra Pradesh and Delhi. Gujarat and Haryana also contribute a significant number of hospitals to the dataset. States such as Assam, Bihar, Chandigarh, and Chhattisgarh have comparatively fewer hospitals.